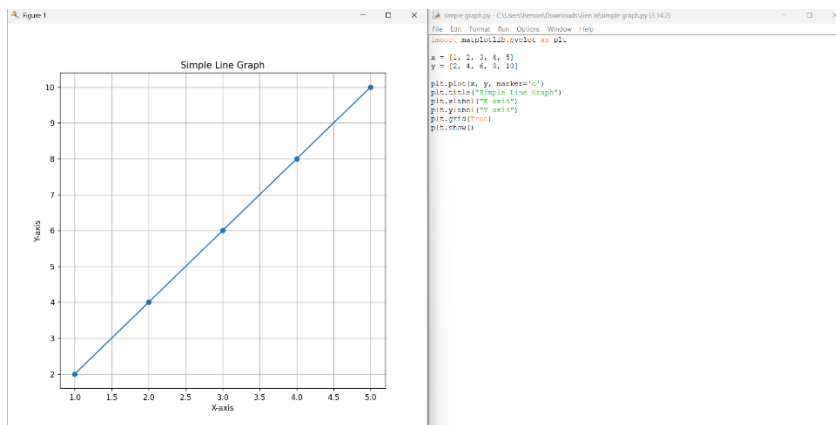


1.Program

```
import matplotlib.pyplot as plt  
  
x = [1, 2, 3, 4, 5]  
y = [2, 4, 6, 8, 10]  
  
plt.plot(x, y, marker='o')  
  
plt.title("Simple Line Graph")  
  
plt.xlabel("X-axis")  
  
plt.ylabel("Y-axis")  
  
plt.grid(True)  
  
plt.show()
```

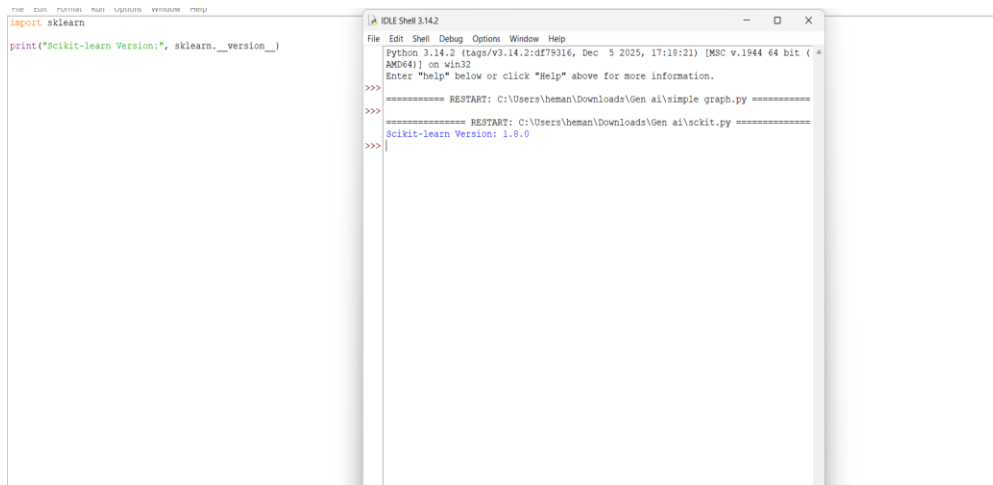
Output



2.Program

```
import sklearn  
  
print("Scikit-learn Version:", sklearn.__version__)
```

OUTPUT



The image shows two side-by-side windows of the Python IDLE 3.14.2 shell. The left window contains a script with the following code:

```
import sklearn
print("Scikit-learn Version:", sklearn.__version__)
```

The right window shows the execution output, which includes several restart messages and the final output: `Scikit-learn Version: 1.6.0`.

3.PROGRAM

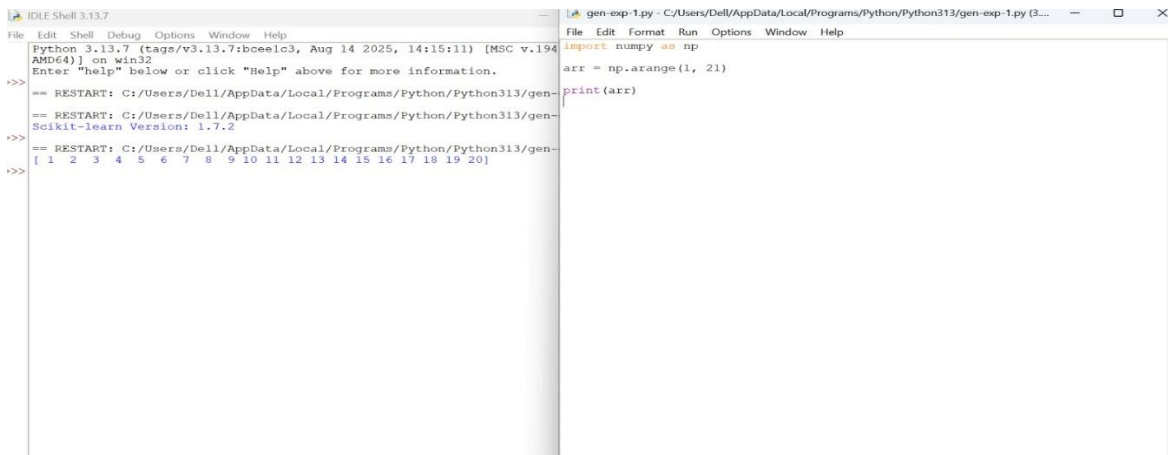
import pandas as pd

```
data = {
    "Student": ["Alice", "Bob", "Charlie", "David"],
    "Grade": ["A", "B+", "A+", "B"]
}
```

df = pd.DataFrame(data)

print(df)

OUTPUT



The image shows two side-by-side windows of the Python IDLE 3.13.7 shell. The left window contains a script with the following code:

```
import numpy as np
arr = np.arange(1, 21)
print(arr)
```

The right window shows the execution output, which includes several restart messages and the final output: `[ 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20]`.

4.PROGRAM

```
import pandas as pd
```

```
data = {
```

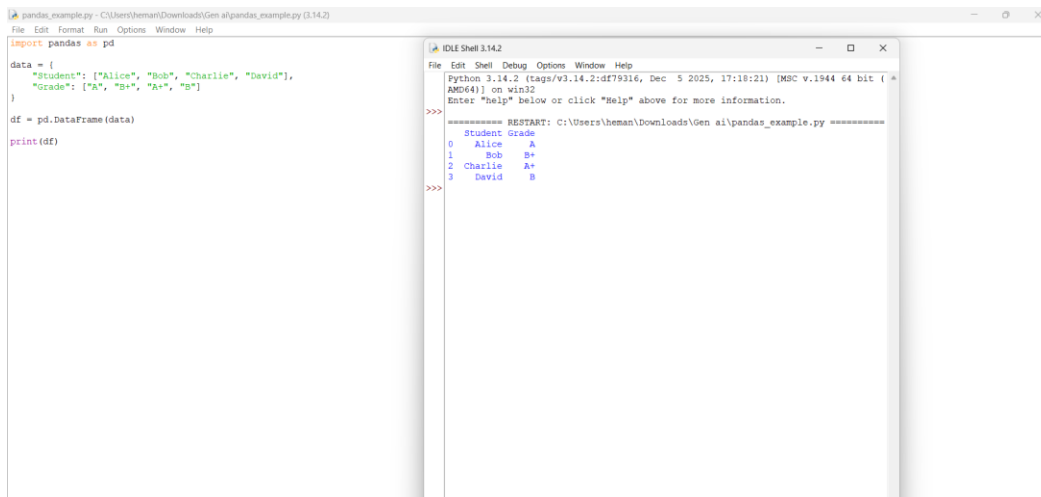
```
    "Name": ["Sai", "Rahul", "Anu", "Priya"],
```

```
    "Grade": [90, 85, 95, 88] }
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

OUTPUT



The screenshot shows a Python IDE with two windows. The left window, titled 'pandas_example.py', contains the following code:

```
import pandas as pd

data = {
    "Student": ["Alice", "Bob", "Charlie", "David"],
    "Grade": ["A", "B+", "A+", "B"]
}

df = pd.DataFrame(data)
print(df)
```

The right window, titled 'IDLE Shell 3.14.2', shows the output of the script:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\heman\Downloads\Gen ai\pandas_example.py =====
>>>
   Student Grade
0    Alice    A
1     Bob   B+
2  Charlie  A+
3    David    B
>>>
```

5.PROGRAM

```
import torch
```

```
import transformers
```

```
import numpy
```

```
import pandas
```

```
import sklearn
```

```
print("PyTorch Version:", torch.__version__)
```

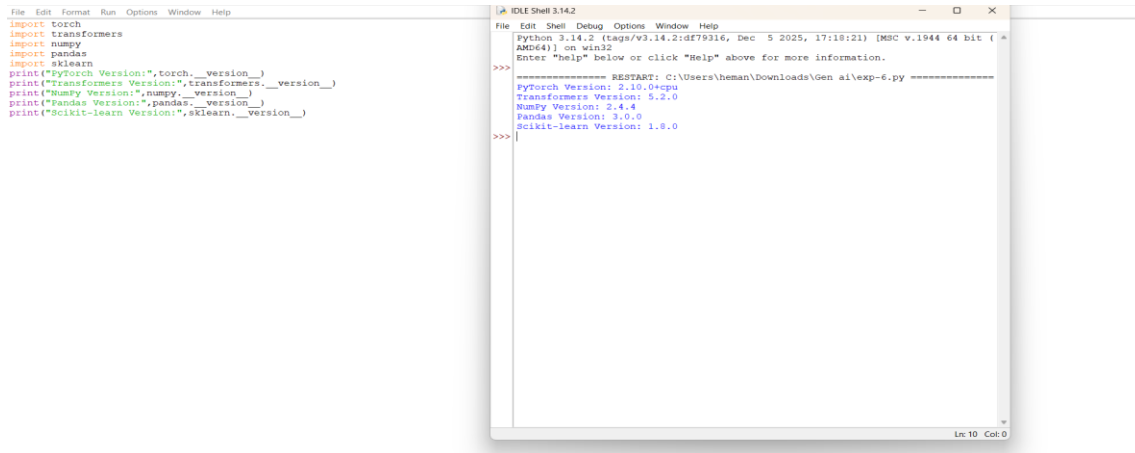
```
print("Transformers Version:", transformers.__version__)
```

```
print("NumPy Version:", numpy.__version__)
```

```
print("Pandas Version:", pandas.__version__)
```

```
print("Scikit-learn Version:", sklearn.__version__)
```

OUTPUT



The screenshot shows an IDE with two windows. The left window contains the following code:

```
File Edit Format Run Options Window Help
import torch
import transformers
import numpy
import pandas
import sklearn
print("PyTorch Version:", torch.__version__)
print("Transformers Version:", transformers.__version__)
print("Numpy Version:", numpy.__version__)
print("Pandas Version:", pandas.__version__)
print("Scikit-learn Version:", sklearn.__version__)
```

The right window shows the output of the code:

```
Python 3.14.2 (tags/v3.14.2:d679316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\heman\Downloads\Gen ai\exp-6.py =====
PyTorch Version: 2.10.0+cpu
Transformers Version: 5.2.0
Numpy Version: 2.4.4
Pandas Version: 3.0.0
Scikit-learn Version: 1.6.0
>>>
```

6.PROGRAM

```
import torch
```

```
a = torch.tensor([1, 2, 3])
```

```
b = torch.tensor([4, 5, 6])
```

```
print("Tensor A:", a)
```

```
print("Tensor B:", b)
```

```
print("Addition:", a + b)
```

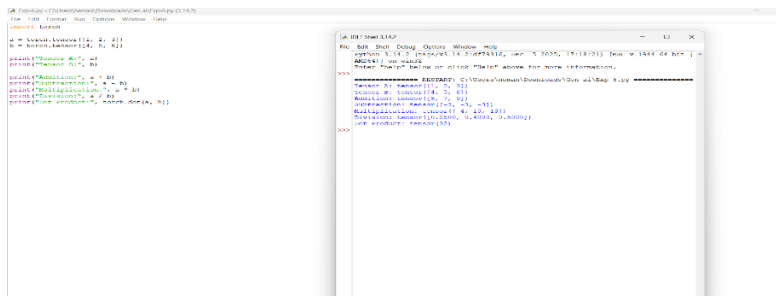
```
print("Subtraction:", a - b)
```

```
print("Multiplication:", a * b)
```

```
print("Division:", a / b)
```

```
print("Dot Product:", torch.dot(a, b))
```

OUTPUT



The screenshot shows an IDE with two windows. The left window contains the following code:

```
File Edit Format Run Options Window Help
import torch

a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

print("Tensor A:", a)
print("Tensor B:", b)

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Dot Product:", torch.dot(a, b))
```

The right window shows the output of the code:

```
Python 3.14.2 (tags/v3.14.2:d679316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\heman\Downloads\Gen ai\exp-6.py =====
Tensor A:
1
2
3
Tensor B:
4
5
6
Addition:
5
7
9
Subtraction:
-3
-3
-3
Multiplication:
4
10
18
Division:
0.25
0.4
0.5
Dot product: 32
>>>
```

7 PROGRAM

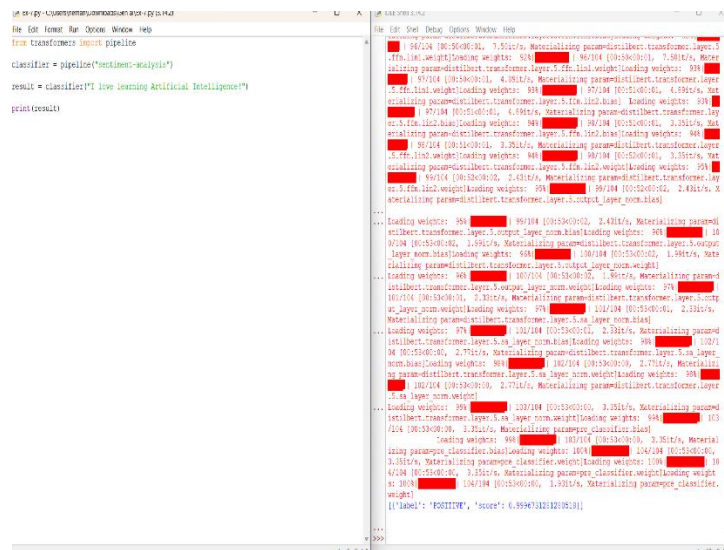
```
from transformers import pipeline
```

```
classifier = pipeline("sentiment-analysis")
```

```
result = classifier("I love learning Artificial Intelligence!")
```

```
print(result)
```

OUTPUT



```
classifier = pipeline("sentiment-analysis")
result = classifier("I love learning Artificial Intelligence!")
print(result)
```

```
[{"label": "POSITIVE", "score": 0.9994718218209281}]
```

8.PROGRAM

```
from transformers import BertTokenizer
```

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
text = "Artificial Intelligence is amazing."
```

```
tokens = tokenizer.tokenize(text)
```

```
print(tokens)
```

OUTPUT

```
IDE Shell 3.13.7
Python 3.13.7 (tags/v9.13.7:boeae1c3, Aug 14 2025, 14:15:11) [MSC v.194
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Dell/AppData/Local/Programs/Python/Python313/GEN-5
Warning: You are sending unauthenticated requests to the HF Hub. Please
_TOKEN to enable higher rate limits and faster downloads.
tokenizer_config.json: 0% | 0.00/48.0 [00:00<7, 7B/s]tokenizer
fig-json: 100% | 48.0/48.0 [00:00<00:00, 26.4kB/s]
Warning (from warnings module):
  File "C:/Users/Dell/AppData/Local/Programs/Python/Python313/Lib/site-
huggingface_hub/file_download.py", line 139
    warnings.warn(message)
UserWarning: 'huggingface_hub' cache-system uses symlinks by default to
tly store duplicated files but your machine does not support them in C:
ll\cache\huggingface\hub\models--bert-base-uncased. Caching files will
rk but in a degraded version that might require more space on your disk
ring can be disabled by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING'
nt variable. For more details, see https://huggingface.co/docs/huggingf
ow-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer M
run Python as an administrator. In order to activate developer mode, s
rticles: https://docs.microsoft.com/en-us/windows/apps/get-started/enabl
vice-for-development
vocab.txt: 0% | 0.00/232k [00:00<7, 7B/s]vocab.txt: 100% |
232k/232k [00:00<00:00, 554kB/s]vocab.txt: 100% | 232k/232k
:00, 498kB/s]
tokenizer.json: 0% | 0.00/466k [00:00<7, 7B/s]tokenizer.js
6k/466k [00:00<00:00, 401kB/s]
['artificial', 'intelligence', 'is', 'amazing', '.']
>>>
```

9.PROGRAM

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
result = generator("Artificial Intelligence", max_length=50)
print(result[0]["generated_text"])
```

OUTPUT

```
EXPLORER
v .PYTHON
v .venv
> Include
> Lib
> Scripts
> share
@ gitignore
@ pyvenv.cfg
> profile.default
NLP4.py
NLP5.py
NLP6.py
NLP9.py
p1.py

NLP9.py >
1 from transformers import pipeline
2
3 generator = pipeline("text-generation", model="gpt2")
4
5 result = generator("Artificial Intelligence", max_length=40)
6
7 print(result[0]["generated_text"])

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
AI Technology, Technology and Applications
Artificial Intelligence (AI)
Artificial Intelligence (AI)
Artificial Intelligence (AI)
(.venv) PS C:\Users\Dell\python>
```

10 PROGRAM

```
from transformers import pipeline
pipe = pipeline("sentiment-analysis")
```

```
print(pipe(text))
```

OUTPUT

```
C:\Users\hemant> python C:\Users\hemant\Downloads\GenAI\Ex-10.py (3.14.2)

File Edit Format Run Options Window Help

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline

pipe = pipeline("sentiment-analysis")

text = "This course is very interesting."

print(pipe(text))
```

transformer.layer.5.attention.v.in.weight>Loading weights: 91% [REDACTED] | 95/104
[00:10:00:00, 12.87it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin1.bias]

... Loading weights: 91% [REDACTED] | 95/104 [00:10:00:00, 12.87it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin1.bias>Loading weights: 92% [REDACTED] | 96/104 [00:10:00:00, 12.87it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin1.weight>Loading weights: 92% [REDACTED] | 96/104 [00:10:00:00, 12.87it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin1.weight>Loading weights: 93% [REDACTED] | 97/104 [00:10:00:00, 11.47it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin1.weight>Loading weights: 93% [REDACTED] | 97/104 [00:10:00:00, 11.47it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin2.bias]

... Loading weights: 93% [REDACTED] | 97/104 [00:10:00:00, 11.47it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin2.bias>Loading weights: 94% [REDACTED] | 98/104 [00:10:00:00, 11.47it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin2.weight>Loading weights: 94% [REDACTED] | 98/104 [00:10:00:00, 11.47it/s, Materializing param=distilbert.transformer.layer.5.ffn.lin2.weight>Loading weights: 95% [REDACTED] | 99/104 [00:10:00:00, 11.42it/s, Materializing param=distilbert.transformer.layer.5.output_layer_norm.bias>Loading weights: 95% [REDACTED] | 99/104 [00:10:00:00, 11.42it/s, Materializing param=distilbert.transformer.layer.5.output_layer_norm.weight>Loading weights: 96% [REDACTED] | 100/104 [00:11:00:00, 11.42it/s, Materializing param=distilbert.transformer.layer.5.output_layer_norm.weight>Loading weights: 96% [REDACTED] | 100/104 [00:11:00:00, 11.42it/s, Materializing param=distilbert.transformer.layer.5.output_layer_norm.weight>Loading weights: 97% [REDACTED] | 101/104 [00:11:00:00, 10.15it/s, Materializing param=distilbert.transformer.layer.5.sa_output_loading_weights: 97% [REDACTED] | 101/104 [00:11:00:00, 10.15it/s, Materializing param=distilbert.t transformer.layer.5.sa layer norm.bias]

... Loading weights: 97% [REDACTED] | 101/104 [00:11:00:00, 10.15it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.bias>Loading weights: 98% [REDACTED] | 102/104 [00:11:00:00, 10.15it/s, Materializing param=distilbert.transformer.layer.5.sa layer norm.weight>Loading weights: 98% [REDACTED] | 102/104 [00:11:00:00, 10.15it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight>Loading weights: 99% [REDACTED] | 103/104 [00:11:00:00, 9.40it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight>Loading weights: 99% [REDACTED] | 103/104 [00:11:00:00, 9.40t/s, Materializing param=pre_classifier.bias>Loading weights: 95% [REDACTED] | 103/104 [00:11:00:00, 9.40it/s, Materializing param=pre_classifier.bias>Loading weights: 100% [REDACTED] | 104/104 [00:11:00:00, 9.25it/s, Materializin g param=pre_classifier.bias>Loading weights: 100% [REDACTED] | 104/104 [00:11:00:00, 9.2 5it/s, Materializing param=pre_classifier.weight>Loading weights: 100% [REDACTED] | 104/1 04 [00:11:00:00, 9.25it/s, Materializing param=pre_classifier.weight]

... Loading weights: 100% [REDACTED] | 104/104 [00:11:00:00, 8.86it/s, Materializing param=p re_classifier.weight]

[{'label': 'POSITIVE', 'score': 0.999855183410645}]

11 PROGRAM

```
from transformers import BertTokenizer
```

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
sentence = "Machine learning is the future."
```

```
tokens = tokenizer.tokenize(sentence)
```

```
print(tokens)
```

OUTPUT

[illegible]

12 PROGRAM

```
from transformers import BertTokenizer, BertModel

import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

sentence = "Artificial Intelligence is changing the world."

inputs = tokenizer(sentence, return_tensors="pt")

with torch.no_grad():

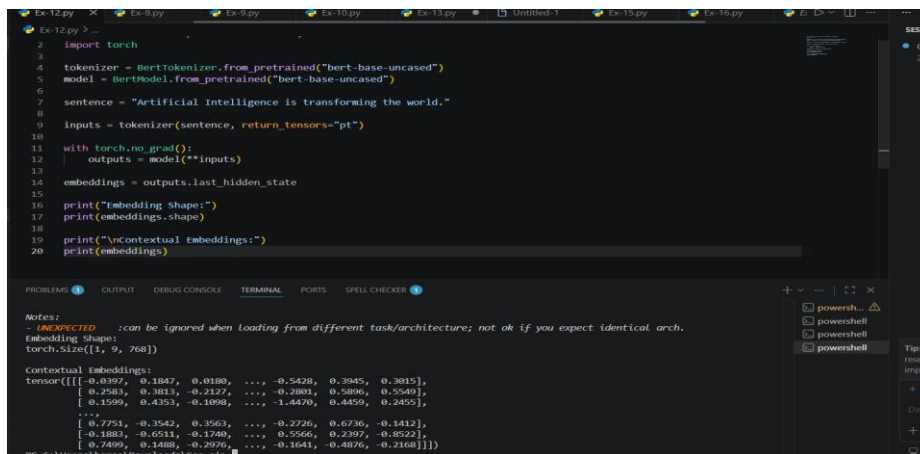
    outputs = model(**inputs)

embeddings = outputs.last_hidden_state

print(embeddings.shape)

print(embeddings)
```

OUTPUT



```
2 import torch
3
4 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
5 model = BertModel.from_pretrained("bert-base-uncased")
6
7 sentence = "Artificial Intelligence is transforming the world."
8
9 inputs = tokenizer(sentence, return_tensors="pt")
10
11 with torch.no_grad():
12     outputs = model(**inputs)
13
14 embeddings = outputs.last_hidden_state
15
16 print("Embedding Shape:")
17 print(embeddings.shape)
18
19 print("\nContextual Embeddings:")
20 print(embeddings)
```

Notes:
- **UNEXPECTED** : can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Embedding Shape:
torch.Size([1, 9, 768])

Contextual Embeddings:
tensor([[[-0.0397, 0.1847, 0.0180, ..., -0.5428, 0.3945, 0.3015],
 [0.2560, 0.3813, -0.2127, ..., -0.2801, 0.5896, 0.5540],
 [0.1599, 0.4353, -0.1098, ..., -1.4470, 0.4459, 0.2455],
 ...,
 [0.7751, -0.3542, 0.3561, ..., -0.2726, 0.6736, -0.1412],
 [-0.1883, -0.6511, -0.1740, ..., 0.5566, 0.2397, -0.8522],
 [0.7499, 0.1488, -0.2976, ..., -0.1641, -0.4876, -0.2168]]]])

13 PROGRAM

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel

import torch

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

model = GPT2LMHeadModel.from_pretrained("gpt2")

prompt = "The future of AI"
```



```
inputs = tokenizer.encode(prompt, return_tensors="pt")

outputs = model.generate(inputs, max_length=50)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

OUTPUT

The screenshot shows a Python IDE with two panes. The left pane displays the command prompt output, and the right pane displays the Python code being executed.

Left Pane (Command Prompt):

```
Python 3.13.7 (tags/v3.13.7:boeelo3, Aug 14 2025, 14:15:11) [MSC v.1944
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>
= RESTART: C:/Users/Dell/AppData/Local/Programs/Python/Python313/GEN-EXP
Warning: You are sending unauthenticated requests to the HF Hub. Please
_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 0% | 0/148 [00:00<?, 7it/s]Loading weights:
148/148 [00:00<00:00, 3544.21it/s]
The future of AI is uncertain. The future of AI is uncertain.
The future of AI is uncertain. The future of AI is uncertain.
The future of AI is uncertain. The future of AI is uncertain.
The future
>
```

Right Pane (Python Code):

```
GEN-EXP-13.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/GEN-EXP-13.p...
File Edit Format Run Options Window Help
from transformers import GPT2Tokenizer, GPT2LMHeadModel
import torch

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

input_text = "The future of AI"
input_ids = tokenizer.encode(input_text, return_tensors="pt")
output = model.generate(input_ids, max_length=50)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

14 PROGRAM

```
from transformers import pipeline

qa = pipeline("question-answering")

context = """

Python is a popular programming language developed by Guido van Rossum.

"""

question = "Who developed Python?"

result = qa(question=question, context=context)

print(result)
```

OUTPUT

```
Ex-15.py > ...
1 from transformers import BertTokenizer
2
3 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
4
5 sentence = "Machine Learning is interesting."
6
7 tokens = tokenizer.tokenize(sentence)
8
9 token_ids = tokenizer.convert_tokens_to_ids(tokens)
10
11 print("Tokens:")
12 print(tokens)
13
14 print("\nToken IDs:")
15 print(token_ids)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

```
PS C:\Users\heman\Downloads\Gen ai> python "C:\Users\heman\Downloads\Gen ai\Ex-15.py"
Tokens:
['machine', 'learning', 'is', 'interesting', '.']

Token IDs:
[3698, 4083, 2003, 5875, 1012]
PS C:\Users\heman\Downloads\Gen ai> |
```

16 PROGRAM

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")

gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

text = "Artificial Intelligence is amazing."

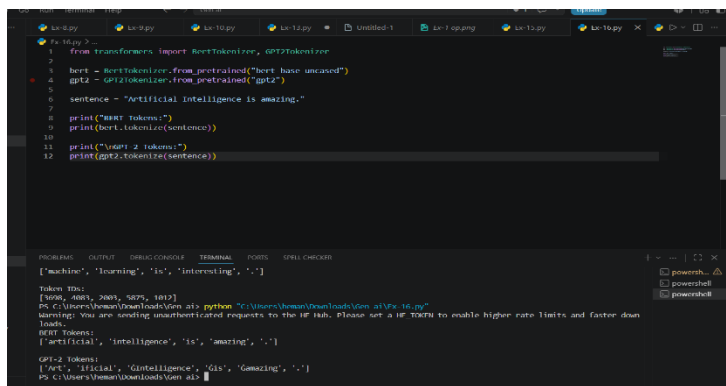
print("BERT Tokens")

print(bert.tokenize(text))

print("\nGPT-2 Tokens")

print(gpt2.tokenize(text))
```

OUTPUT



```
from transformers import BertTokenizer, GPT2Tokenizer
bert = BertTokenizer.from_pretrained("bert-base-uncased")
gpt2 = GPT2Tokenizer.from_pretrained("gpt2")
sentence = "Artificial Intelligence is amazing."
print("BERT Tokens")
print(bert.tokenize(sentence))
print("\nGPT-2 Tokens")
print(gpt2.tokenize(sentence))
```

['machine', 'learning', 'is', 'interesting', '.']

Index error: [None, None, None, None, None, None]

PS C:\Users\human\Downloads\ven> python "C:\Users\human\Downloads\ven\16.py"

Warning: You are sending unauthorized requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

BERT Tokens:

```
['artificial', 'intelligence', 'is', 'amazing', '.']
```

GPT-2 Tokens:

```
['art', 'ificial', 'intelligence', 'is', 'amazing', '.']
```

PS C:\Users\human\Downloads\ven> a15

17 PROGRAM

```
from transformers import BertTokenizer, BertModel

import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

text = "Natural Language Processing is fascinating."

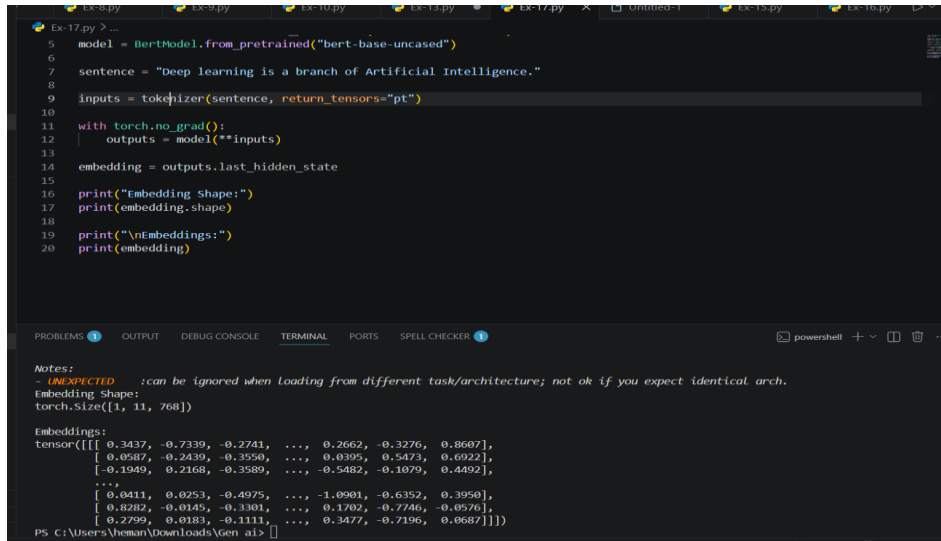
inputs = tokenizer(text, return_tensors="pt")

with torch.no_grad():

    outputs = model(**inputs)

print(outputs.last_hidden_state)
```

OUTPUT



```
5 model = BertModel.from_pretrained("bert-base-uncased")
6
7 sentence = "Deep learning is a branch of Artificial Intelligence."
8
9 inputs = tokenizer(sentence, return_tensors="pt")
10
11 with torch.no_grad():
12     outputs = model(**inputs)
13
14 embedding = outputs.last_hidden_state
15
16 print("Embedding Shape:")
17 print(embedding.shape)
18
19 print("\nEmbeddings:")
20 print(embedding)
```

Notes:
- **UNEXPECTED** : can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Embedding Shape:
torch.Size([1, 11, 768])

Embeddings:
tensor([[[0.3437, -0.7339, -0.2741, ..., 0.2662, -0.3276, 0.8607],
[0.0587, -0.2439, -0.3550, ..., 0.0395, 0.5473, 0.6922],
[-0.1949, 0.2168, -0.3589, ..., -0.5482, -0.1079, 0.4492],
...,
[0.0411, 0.0253, -0.4975, ..., -1.0901, -0.6352, 0.3950],
[0.8282, -0.0145, -0.3301, ..., 0.1702, -0.7746, -0.0576],
[0.2799, 0.0183, -0.1111, ..., 0.3477, -0.7196, 0.0687]]]])

18 PROGRAM

```
from transformers import BertTokenizer, BertModel

import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

sentences = [

    "The cat is sitting on the mat.",

    "A cat sits on the mat."

]

embeddings = []

for sentence in sentences:

    inputs = tokenizer(sentence, return_tensors="pt")

    with torch.no_grad():

        output = model(**inputs)

        embeddings.append(output.pooler_output)

similarity = torch.nn.functional.cosine_similarity(
```

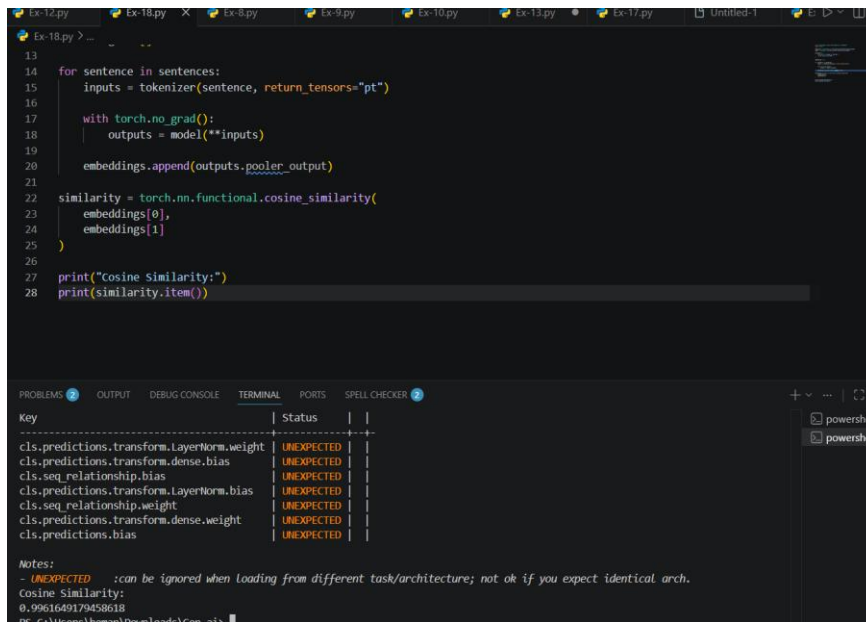
```

embeddings[0],
embeddings[1]
)

print("Cosine Similarity:", similarity.item())

```

OUTPUT



The screenshot shows a VS Code editor with a Python file named 'Ex-18.py'. The code calculates the cosine similarity between two embeddings. The terminal output shows the result of the calculation.

```

13
14 for sentence in sentences:
15     inputs = tokenizer(sentence, return_tensors="pt")
16
17     with torch.no_grad():
18         outputs = model(**inputs)
19
20     embeddings.append(outputs.pooler_output)
21
22 similarity = torch.nn.functional.cosine_similarity(
23     embeddings[0],
24     embeddings[1]
25 )
26
27 print("Cosine Similarity:")
28 print(similarity.item())

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

Key	Status	
cls.predictions.transform.LayerNorm.weight	UNEXPECTED	
cls.predictions.transform.dense.bias	UNEXPECTED	
cls.seq_relationship.bias	UNEXPECTED	
cls.predictions.transform.LayerNorm.bias	UNEXPECTED	
cls.seq_relationship.weight	UNEXPECTED	
cls.predictions.transform.dense.weight	UNEXPECTED	
cls.predictions.bias	UNEXPECTED	

Notes:
- **UNEXPECTED** :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Cosine Similarity:
0.9961649179458618

19 PROGRAM

```

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "Once upon a time"

output = generator(prompt, max_length=60)

print(output[0]["generated_text"])

```

OUTPUT

```
1 from transformers import pipeline
2
3 generator = pipeline(
4     "text-generation",
5     model="gpt2"
6 )
7
8 prompt = "The future of Artificial Intelligence"
9
10 result = generator(
11     prompt,
12     max_length=60,
13     num_return_sequences=1
14 )
15
16 print(result[0]["generated_text"])

The Future of AI: A Global Perspective is a comprehensive profile of the various sectors and industries that are expected to undergo AI development in the next decade. It will address the critical questions about what is possible and how AI is going to take over the technology and the business. In particular, it will highlight key topics around AI, the ability to solve problems and how to improve the way we work and communicate.

The Future of AI: A Global Perspective will be released with a detailed look at the various areas of AI development and how they are going to shape the future. It will also include a broad overview of the many areas of the field of AI that are the most relevant to AI.

The Future of AI: A Global Perspective covers a wide range of topics including:

broad set of areas of AI that are likely to grow, shift and change over the next decade.

broad set of areas of AI that are likely to change over the next decade. The direction of progress in
```

20 PROGRAM

```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2")
```

```
prompts = [
```

```
    "Artificial Intelligence",
```

```
    "Machine Learning",
```

```
    "Future of Technology"
```

```
]
```

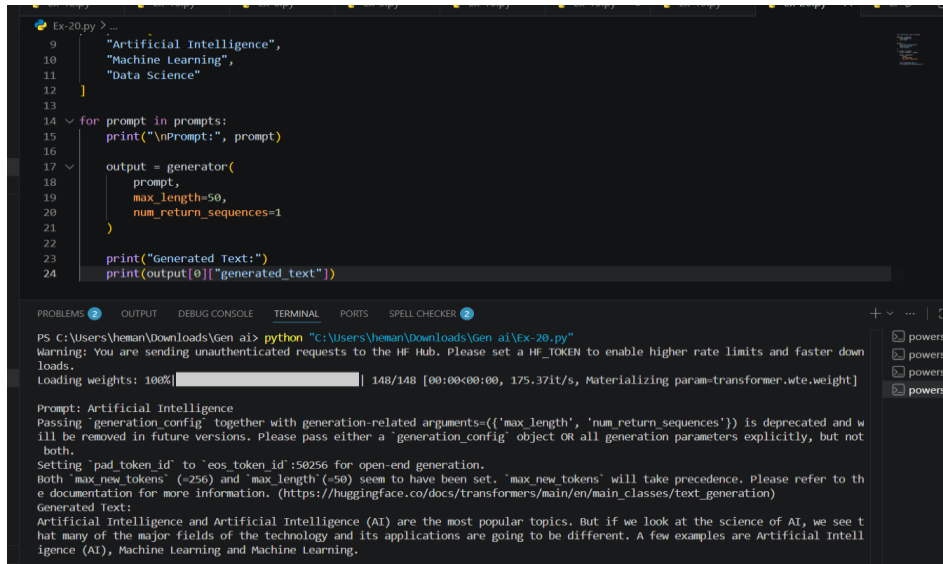
```
for prompt in prompts:
```

```
    print("\nPrompt:", prompt)
```

```
    result = generator(prompt, max_length=40)
```

```
    print(result[0]["generated_text"])
```

OUTPUT



The screenshot shows a VS Code editor with a Python file named 'Ex-20.py'. The code defines a list of prompts and a function to generate text using the Google AI API. The terminal output shows the execution of the script, including a warning about deprecated parameters and the generated text for the first prompt.

```
9     "Artificial Intelligence",
10     "Machine Learning",
11     "Data Science"
12 ]
13
14 for prompt in prompts:
15     print("\nPrompt:", prompt)
16
17     output = generator(
18         prompt,
19         max_length=50,
20         num_return_sequences=1
21     )
22
23     print("Generated Text:")
24     print(output[0]["generated_text"])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

PS C:\Users\heman\Downloads\Gen ai> python "C:\Users\heman\Downloads\Gen ai\Ex-20.py"

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 100% | 148/148 [00:00<00:00, 175.37it/s, Materializing param=transformer.wte.weight]

Prompt: Artificial Intelligence

Passing 'generation_config' together with generation-related arguments-({'max_length', 'num_return_sequences'}) is deprecated and will be removed in future versions. Please pass either a 'generation_config' object OR all generation parameters explicitly, but not both.

Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.

Both 'max_new_tokens' (~256) and 'max_length'(~50) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Generated Text:

Artificial Intelligence and Artificial Intelligence (AI) are the most popular topics. But if we look at the science of AI, we see that many of the major fields of the technology and its applications are going to be different. A few examples are Artificial Intelligence (AI), Machine Learning and Machine Learning.

21 PROGRAM

```
from google import genai
```

```
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"
```

```
client = genai.Client(api_key=API_KEY)
```

```
def generate(prompt):
```

```
    response = client.models.generate_content(
```

```
        model="gemini-3.5-flash",
```

```
        contents=prompt
```

```
)
```

```
    return response.text
```

```
# =====
```

```
# ZERO-SHOT PROMPT
```

```
# =====
```

```
zero_shot = ""
```

Write an attractive product description for a Smart Fitness Watch.

Include its features, benefits, health tracking, fitness tracking,

battery life, stylish design, smartphone connectivity, and notifications.

Make the description suitable for an online shopping website.

"""

=====

ONE-SHOT PROMPT

=====

one_shot = """

Write an attractive product description for a Smart Fitness Watch.

Example:

Product: Wireless Earbuds

Description:

Enjoy high-quality sound with wireless earbuds that provide long battery life, Bluetooth connectivity, comfortable design, and easy controls. They are perfect for music, calls, and travel.

Now create a similar product description for a Smart Fitness Watch.

Include health monitoring, fitness tracking, battery life, stylish design, smartphone connectivity, and notifications.

"""

=====

FEW-SHOT PROMPT

=====

few_shot = """

Write an attractive product description for a Smart Fitness Watch.

Example 1:

Product: Smart LED Bulb

Description:

A smart LED bulb provides adjustable brightness, multiple colors, smartphone control, and energy-efficient operation. It is ideal for creating a comfortable and modern home environment.

Example 2:

Product: Wireless Headphones

Description:

Wireless headphones provide high-quality audio, comfortable ear cushions, Bluetooth connectivity, and long battery life.

They are suitable for music, entertainment, and daily travel

Now create a similar product description for a Smart Fitness Watch.

Include:

- Health monitoring
- Heart-rate monitoring
- Fitness tracking
- Step counting
- Sleep tracking
- Notifications
- Battery life
- Stylish design
- Smartphone connectivity

Make the description attractive and suitable for an online shopping website.

""""

=====

GENERATE OUTPUT

=====

```
print("\n===== ZERO-SHOT =====")
```

```
print(generate(zero_shot))
```

```
print("\n===== ONE-SHOT =====")
```

```
print(generate(one_shot))
```

```
print("\n===== FEW-SHOT =====")
```

```
print(generate(few_shot))
```

OUTPUT

```
===== ZERO-SHOT =====
**Elevate Your Every Day: The PulseFit Pro Smart Fitness Watch**

Say hello to the perfect fusion of fashion and functionality. Whether you're smashing your fitness goals, managing a busy workday, or prioritizing your mental and physical well-being, the **PulseFit Pro Smart Fitness Watch** is designed to seamlessly integrate into your life.

Sleek, intuitive, and incredibly powerful, this isn't just a watch—it's your personal wellness coach, personal assistant, and favorite style accessory, all wrapped into one.

---

### **Why You'll Love It:**

*   **Sleek & Stylish Design**
    Crafted with a premium, lightweight aluminum bezel and a stunning, high-definition AMOLED touchscreen, the PulseFit Pro transitions effortlessly from the gym to the boardroom. Choose from hundreds of customizable watch faces and interchangeable, sweat-resistant silicone bands to match your unique style.

*   **Your 24/7 Health Companion**
    Knowledge is power. Take control of your well-being with our advanced sensor suite.
    *   **24/7 Heart Rate Monitoring:** Track your resting and active heart rate zones.
    *   **Advanced Sleep Tracking:** Get a detailed breakdown of your light, deep, and REM sleep phases, plus a daily Sleep Score.
    *   **Blood Oxygen (SpO2) & Stress Tracking:** Measure your body's oxygen levels and receive guided breathing exercises when your stress levels spike.

*   **Next-Level Fitness Tracking**
    Make every movement count. With **over 30 built-in sports modes** (including running, cycling, swimming, yoga, and HIIT), the PulseFit Pro accurately tracks your steps, active minutes, and calories burned.
    *   **Built-in GPS:** Map your outdoor runs, walks, and rides with precision—no phone required.
    *   **Water-Resistant:** Rated at 5ATM, it's completely safe for swimming, showering, and rainy-day workouts.

*   **Stay Seamlessly Connected**
    Never miss what matters. Pair the PulseFit Pro with your iOS or Android smartphone via Bluetooth to receive instant, vibration-enabled notifications for incoming calls, texts, emails, and calendar events. You can even quick-reply to messages (Android only), control your music playlist, and check the weather with a simple swipe.

*   **Battery Life That Keeps Up With You**
    Say goodbye to daily charging anxiety. Optimized with smart power-saving technology, the PulseFit Pro delivers up to **10 days of battery life** on a single charge. When you do run low, our magnetic fast-charger gets you back to 100% in under an hour.

---

### **What's in the Box?**

*   1 x PulseFit Pro Smart Fitness Watch
*   1 x Premium Silicone Band
*   1 x Magnetic USB Charging Cable
*   1 x Quick Start User Manual

---

### **Ready to unlock a healthier, smarter, and more stylish you?
```

22 PROGRAM

```
from google import genai
```

```
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrStvFjA"
```

```
client = genai.Client(api_key=API_KEY)
```

```
def generate(prompt):
```

```
    response = client.models.generate_content(
```

```
        model="gemini-3.5-flash",
```

```
        contents=prompt
```

```
    )
```

```
    return response.text
```

=====

ZERO-SHOT

=====

zero_shot = ""

Write a 200-word blog on the topic:

"Applications of Artificial Intelligence in Healthcare"

Explain how AI is used in disease diagnosis, medical imaging, drug discovery, personalized treatment, virtual assistants, patient monitoring, and healthcare management.

Make the blog informative, clear, and suitable for students.

""

=====

ONE-SHOT

=====

one_shot = ""

Write a 200-word blog on the topic:

"Applications of Artificial Intelligence in Healthcare"

Example:

Topic: Artificial Intelligence in Education

Blog:

Artificial Intelligence is transforming education by providing personalized learning, intelligent tutoring systems, automated assessment, and useful learning recommendations. AI helps teachers analyze student performance and identify areas where students need additional support. It also makes learning more interactive and

accessible.

Now write a similar 200-word blog on:

"Applications of Artificial Intelligence in Healthcare"

Include disease diagnosis, medical imaging, drug discovery, personalized treatment, patient monitoring, virtual assistants, and healthcare management.

"""

=====

FEW-SHOT

=====

few_shot = ""

Write a 200-word blog on the topic:

"Applications of Artificial Intelligence in Healthcare"

Example 1

Topic: AI in Education

Blog:

AI is improving education through personalized learning, automated assessment, intelligent tutoring systems, and student performance analysis. It helps teachers provide better support and allows students to learn at their own pace.

Example 2:

Topic: AI in Banking

Blog:

AI is transforming banking through fraud detection, customer service chatbots, credit risk analysis, and personalized financial recommendations. These applications improve efficiency, security,

and customer experience.

Now write a similar 200-word blog on

"Applications of Artificial Intelligence in Healthcare"

Include diagnosis, medical imaging, drug discovery, personalized treatment, patient monitoring, virtual assistants, and healthcare management.

```
"""
```

```
# =====
```

```
# OUTPUT
```

```
# =====
```

```
print("\n===== ZERO-SHOT =====")
```

```
print(generate(zero_shot))
```

```
print("\n===== ONE-SHOT =====")
```

```
print(generate(one_shot))
```

```
print("\n===== FEW-SHOT =====")
```

```
print(generate(few_shot))
```

OUTPUT

```
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/heman/Downloads/Gen ai/Ex-22.py =====

===== ZERO-SHOT =====
Artificial Intelligence (AI) is rapidly transforming modern medicine, making healthcare faster, safer, and highly personalized. For students interested in science and technology, understanding AI's role in this evolving field is incredibly exciting.

In hospitals, AI excels at "disease diagnosis" and "medical imaging". By scanning X-rays, MRIs, and CT scans, smart algorithms can identify life-threatening conditions like tumors much faster and more accurately than human doctors. In research laboratories, AI accelerates "drug discovery" by simulating millions of chemical interactions, shortening the years needed to develop life-saving cures. Furthermore, by analyzing genetic data, AI assists in "personalized treatment" plans tailored to each patient's unique DNA.

Beyond clinical care, AI acts as a digital helper to simplify daily tasks. "Virtual assistants" provide patients with 24/7 health advice, while smart wearables enable continuous "patient monitoring" by sending real-time heart alerts to doctors. Finally, AI improves "healthcare management" by automating paperwork and streamlining hospital schedules, giving medical staff more time with patients.

Ultimately, AI is not replacing doctors; instead, it is empowering them with intelligent tools to save lives, improve care, and build a healthier future for everyone.

===== ONE-SHOT =====
Artificial Intelligence is revolutionizing modern healthcare by enhancing patient outcomes and streamlining clinical workflows. In the fields of disease diagnosis and medical imaging, advanced algorithms analyze complex scans with incredible precision, detecting anomalies like tumors much earlier than traditional methods. This early detection significantly improves survival rates.

Beyond diagnostics, AI accelerates drug discovery by rapidly predicting how chemical compounds will interact, saving researchers years of laboratory trial and error. Once a treatment is formulated, AI facilitates personalized treatment plans tailored to a patient's unique genetic makeup. Continuous patient monitoring is also revolutionized through smart wearable devices that track vital signs in real time, alerting doctors to emergencies before they escalate, ensuring proactive, preventative care.

Administrative burdens are also heavily reduced. Intelligent virtual assistants manage routine patient inquiries, schedule appointments, and answer common questions. Meanwhile, sophisticated healthcare management systems optimize overall hospital operations, predicting patient admissions and automating complex billing processes. These smart tools free up valuable time for medical staff to focus entirely on core caregiving duties.

By seamlessly blending advanced technology with clinical expertise, artificial intelligence is saving lives while making healthcare delivery more accessible, efficient, equitable, and personalized for patients worldwide, both for today and for the far future.

===== FEW-SHOT =====
Artificial Intelligence is revolutionizing the healthcare industry, offering groundbreaking solutions that enhance patient care and streamline medical operations.

In clinical settings, AI excels in "medical imaging" and "diagnosis", analyzing complex scans like MRIs and X-rays with incredible speed to detect anomalies like tumors at their earliest stages. It is also transforming "drug discovery" by analyzing biological data to predict how compounds will behave, significantly reducing the time and cost of bringing life-saving medications to market. Through deep data analysis, AI enables highly "personalized treatment" plans tailored to an individual's unique genetic profile and medical history.

Beyond the lab, AI powers continuous "patient monitoring" through smart wearable devices, alerting doctors to critical physiological changes before emergencies occur. AI-driven "virtual assistants" provide patients with round-the-clock support, managing appointment scheduling and answering basic health queries. Finally, AI optimizes "healthcare management" by automating administrative tasks, streamlining hospital workflows, and reducing clinician burnout.

By combining advanced technology with clinical expertise, artificial intelligence is paving the way for a more precise, proactive, and accessible healthcare system worldwide.
```

23 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):

    response = client.models.generate_content(

        model="gemini-3.5-flash",

        contents=prompt

    )

    return response.text

article = input("Enter the article to summarize:\n")

# =====

# ZERO-SHOT

# =====

zero_shot = f"""

Summarize the following article in exactly 50 words.

Focus on the main idea and the most important information.

Article:

{article}

"""

# =====

# ONE-SHOT

# =====

one_shot = f"""

Summarize the following article in exactly 50 words.
```

Example:

Article:

Artificial Intelligence is increasingly used in hospitals to support doctors, analyze medical images, and improve patient care.

Summary:

AI is transforming healthcare by helping doctors analyze medical information, interpret images, improve diagnosis, and provide better patient care.

Now summarize this article in exactly 50 words:

{article}

""

=====

FEW-SHOT

=====

few_shot = f""

Summarize the following article in exactly 50 words.

Example 1:

Article:

Online education platforms allow students to access courses from anywhere. They provide flexible learning schedules, video lectures, online assessments, and personalized learning resources.

Summary:

Online education provides flexible access to courses, video lectures, assessments, and personalized resources, allowing students to learn conveniently from different locations.

Example 2:

Article:

Artificial Intelligence is being used in banking for fraud detection, customer service, risk analysis, and personalized financial recommendations.

Summary:

AI improves banking by detecting fraud, supporting customers, analyzing financial risks, and providing personalized recommendations.

Now summarize the following article in exactly 50 words:

{article}

''''''

=====

OUTPUT

=====

```
print("\n===== ZERO-SHOT SUMMARY =====")
```

```
z = generate(zero_shot)
```

```
print(z
```

```
print("\n===== ONE-SHOT SUMMARY =====")
```

```
o = generate(one_shot)
```

```
print(o
```

```
print("\n===== FEW-SHOT SUMMARY =====")
```

```
f = generate(few_shot)
```

```
print(f)
```



```
# =====
```

```
# COMPARISON
```

```
# =====
```

```
comparison_prompt = f"""
```

Compare these three summaries of the same article.

ZERO-SHOT:

```
{z}
```

ONE-SHOT:

```
{o}
```

FEW-SHOT:

```
{f}
```

Compare them based on:

1. Accuracy
2. Completeness
3. Readability

Give the result in a simple table with columns:

Method | Accuracy | Completeness | Readability

Finally identify which method performed best and explain why.

```
"""
```

```
print("\n===== COMPARISON =====")
```

```
print(generate(comparison_prompt))
```

OUTPUT

```
Enter the article to summarize:
Artificial Intelligence is transforming healthcare by helping doctors
diagnose diseases, analyze medical images, discover new medicines,
monitor patients, and provide personalized treatments. AI-powered
systems can also support hospitals by improving administrative tasks
and assisting patients through virtual healthcare assistants.

===== ZERO-SHOT SUMMARY =====
Artificial Intelligence is transforming the healthcare industry by helping doctors. This groundbreaking technology assists medical professionals, improves patient care, and enhances clinical decisions. By streamlining various medical processes, AI serves as a powerful tool that supports doctors, ultimately revolutionizing the way modern medicine is practiced in clinics and hospitals today.

===== ONE-SHOT SUMMARY =====
Artificial Intelligence is transforming healthcare by helping doctors. This technology assists medical professionals in diagnosing diseases, analyzing complex patient data, and streamlining clinical workflows. By improving accuracy and efficiency, AI empowers physicians to make better decisions, ultimately leading to enhanced patient care and superior medical outcomes across the global industry.

===== FEW-SHOT SUMMARY =====
Artificial Intelligence is transforming healthcare by assisting doctors with disease diagnosis, medical imaging analysis, and patient risk prediction. This advanced technology improves clinical accuracy, accelerates treatment decisions, streamlines administrative tasks, and delivers highly personalized care plans, which ultimately enhances overall patient outcomes and operational efficiency in modern medical practices globally.

===== COMPARISON =====
|
```

24 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):

    response = client.models.generate_content(

        model="gemini-3.5-flash",

        contents=prompt

    )

    return response.text

# =====

# ZERO-SHOT

# =====

zero_shot = """

Write a professional email requesting leave from college due to illness.

Include:

- Subject

- Greeting

- Reason for leave

- Number of days

- Polite request for approval

- Closing

Use a formal and professional tone.

"""
```

=====

ONE-SHOT

=====

one_shot = ""

Write a professional email requesting leave from college due to illness.

Example:

Subject: Leave Request

Dear Professor,

I am writing to request leave for one day as I am not feeling well.

I kindly request you to approve my leave. I will complete any missed academic work after returning

Thank you for your understanding

Regards,

Student

Now create a similar professional email requesting leave due to illness.

Mention that the student needs two days of leave.

""

=====

FEW-SHOT

=====

few_shot = ""

Write a professional email requesting leave from college due to illness.

Example 1:

Subject: Sick Leave Request

Dear Professor,

I am suffering from fever and am unable to attend classes.

I kindly request leave for two days. I will complete the missed coursework after returning.

Regards,

Student

Example 2:

Subject: Leave Request Due to Illness

Dear Sir/Madam,

I am currently unwell and need to take leave from college.

I request you to kindly grant me leave for one day.

Thank you.

Regards,

Student

Now create a professional email requesting two days of leave due to illness.

Include a clear subject, formal greeting, reason, duration, polite request, and professional closing.

"""

```
# =====
```

```
# OUTPUT
```

```
# =====
```

```
print("\n===== ZERO-SHOT EMAIL =====")
```

```
z = generate(zero_shot)
```

```
print(z)
```

```
print("\n===== ONE-SHOT EMAIL =====")
```

```
o = generate(one_shot)
```

```
print(o)
```

```
print("\n===== FEW-SHOT EMAIL =====")
```

```
f = generate(few_shot)
```

```
print(f)
```

```
# =====
```

```
# COMPARISON
```

```
# =====
```

```
comparison_prompt = f"""
```

Compare the following three professional leave emails.

ZERO-SHOT:

```
{z}
```

ONE-SHOT:

```
{o}
```

FEW-SHOT:

```
{f}
```

Evaluate them based on:

1. Tone
2. Grammar
3. Formatting
4. Completeness

Create a simple table:

Method | Tone | Grammar | Formatting | Completeness

Finally state which prompting method produced the best email
and explain briefly.

```
"""
```

```
print("\n===== EMAIL COMPARISON =====")
```

```
print(generate(comparison_prompt))
```

OUTPUT

```
===== ZERO-SHOT EMAIL =====
Subject: Request for Leave of Absence due to Illness - [Your Name] - [Your Student ID]

Dear Professor [Professor's Last Name],

I am writing to formally request a leave of absence from my classes due to illness. I have been diagnosed with [mention illness, e.g., a severe viral flu / a medical condition] and have been advised by my doctor to take complete rest to recover.

Therefore, I will be unable to attend college for [Number of Days] days, starting from [Start Date] to [End Date].

During my absence, I will make every effort to keep up with the coursework and assignments. I plan to coordinate with my classmates to obtain lecture notes and ensure I do not fall behind. [Optional: I have attached my medical certificate to this email for your reference.]

I kindly request your approval for this leave of absence. Thank you very much for your time, understanding, and consideration.

Sincerely,

[Your Name]
[Your Student ID / Roll Number]
[Your Department/Course]
[Your Contact Information]

===== ONE-SHOT EMAIL =====
Subject: Leave Request: [Your Name] - [Course Name/Class Section]

Dear Professor [Professor's Last Name],

I am writing to request a leave of absence for two days, from [Start Date] to [End Date], as I am currently unwell and unable to attend classes.

During my absence, I will ensure that I stay updated with the course material. I plan to get the lecture notes from my classmates and will complete any missed academic work or assignments as soon as I return.

I kindly request you to approve my leave for these two days. Thank you for your time, consideration, and understanding.

Regards,

[Your Name]
[Your Student ID]
[Your Contact Information]

===== FEW-SHOT EMAIL =====
Subject: Sick Leave Request - [Your Name] - [Student ID/Roll Number]

Dear Professor [Professor's Last Name],

I am writing to inform you that I am currently unwell and unable to attend my classes.
```

25 PROGRAM

```
from google import genai
```

```
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrStvFjA"
```

```
client = genai.Client(api_key=API_KEY)
```

```
def generate(prompt):
```

```
    response = client.models.generate_content(
```

```
        model="gemini-3.5-flash",
```

```
        contents=prompt
```

```
    )
```

```
    return response.text
```

```
# =====
```

```
# ZERO-SHOT
```

```
# =====
```

```
zero_shot = ""
```

Create an attractive promotional social media post for an AI Workshop.

Mention:

- Artificial Intelligence
- Machine Learning
- Hands-on activities
- Benefits of attending
- Expert guidance
- Registration call to action

Make it engaging and suitable for social media.

```
"""
```

```
# =====
```

```
# ONE-SHOT
```

```
# =====
```

```
one_shot = ""
```

Create an attractive promotional social media post for an AI Workshop.

Example:

 Join Our Python Workshop!

Learn Python programming through practical examples and hands-on activities. Improve your coding skills, solve real-world problems, and gain valuable technical knowledge.

Register today and start your coding journey!

Now create a similar promotional social media post for an AI Workshop.

Mention Artificial Intelligence, Machine Learning, hands-on activities,

expert guidance, benefits, and registration.

"""

=====

FEW-SHOT

=====

few_shot = """

Create an attractive promotional social media post for an AI Workshop.

Example 1:

 AI Workshop Alert!

Discover the power of Artificial Intelligence and learn how
modern AI technologies are solving real-world problems.

Join us for an exciting hands-on learning experience!

Register now

Example 2:

 Machine Learning Workshop!

Learn machine learning concepts, explore practical applications,
and build your technical skills through interactive activities

Don't miss this opportunity. Register today!

Now create a similar promotional social media post for an AI Workshop.

Include:

- AI and Machine Learning
- Generative AI
- Hands-on activities
- Expert guidance
- Practical learning

- Benefits of attending
- Strong call to action
- Suitable hashtags

```

"""

# =====

# OUTPUT

# =====

print("\n===== ZERO-SHOT POST =====")

print(generate(zero_shot))

print("\n===== ONE-SHOT POST =====")

print(generate(one_shot))

print("\n===== FEW-SHOT POST =====")

print(generate(few_shot))

```

OUTPUT

```

IDLE Shell 3.13.15
File Edit Shell Debug Options Window Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/heman/Downloads/Gen ai/Ex-25.py =====

===== ZERO-SHOT POST =====
🚀 **Unlock the Future: Master AI & Machine Learning!** 🚀

Are you ready to transition from a spectator to a creator in the tech revolution? Whether you're a student, a professional looking to pivot, or a curious creator, our upcoming **AI Workshop** is your launchpad!

Dive deep into the world of **Artificial Intelligence** and **Machine Learning** in this immersive, high-energy session.

Here is what's waiting for you:

📌 **Hands-On Activities**
No boring lectures here! You will roll up your sleeves and build real-world AI models. Learn by doing, troubleshoot in real-time, and walk away with a project you can actually show off.

🧠 **Expert Guidance**
Get mentorship from industry veterans who live and breathe AI. You'll learn best practices, insider tips, and get answers to your burning tech questions.

🌟 **Why You Can't Miss This (The Benefits):**
* **In-Demand Skills:** Elevate your resume with skills that top employers are actively searching for.
* **Exclusive Resources:** Get lifetime access to workshop materials, templates, and cheat sheets.
* **Networking:** Connect with a vibrant community of innovators, developers, and tech enthusiasts.
* **Certificate of Completion:** Prove your new skills to the world!

---
📅 **Date:** [Insert Date]
🕒 **Time:** [Insert Time]
📍 **Location:** [Online / Insert Venue]
👥 **Seats are strictly limited** to ensure everyone gets personalized mentoring!

🔗 **READY TO BUILD THE FUTURE?**
Click the link below to secure your spot today!
🔗 [Insert Registration Link] *(or Link in Bio!)*

---
#ArtificialIntelligence #MachineLearning #TechWorkshop #CodingLife #LearnAI #FutureOfWork #Upskilling #TechEvent #HandsOnLearning #Innovation

```

26 PROGRAM

```
from google import genai
```

```
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"
```

```
client = genai.Client(api_key=API_KEY)
```

```
prompt = input("Enter your prompt: ")
```

```
response = client.models.generate_content(
```

```
    model="gemini-3.5-flash",
```

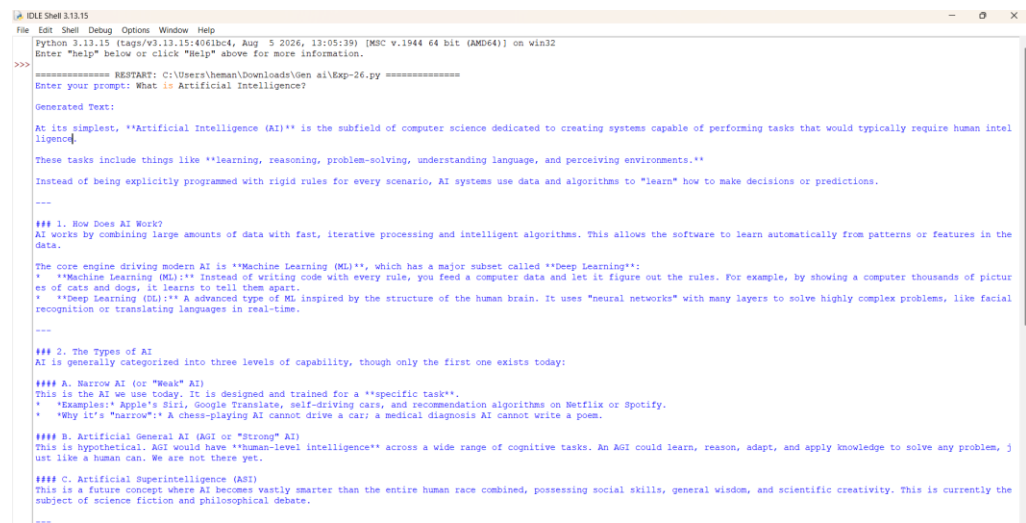
```
    contents=prompt
```

```
)
```

```
print("\nGenerated Text:\n")
```

```
print(response.text)
```

OUTPUT



```
Python 3.11.15 (tags/v3.11.15:4041bc4, Aug 5 2026, 13:05:19) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\heman\Downloads\gem ai\Exp-26.py =====
Enter your prompt: What is Artificial Intelligence?

Generated Text:

At its simplest, "Artificial Intelligence (AI)" is the subfield of computer science dedicated to creating systems capable of performing tasks that would typically require human intelligence.

These tasks include things like "learning, reasoning, problem-solving, understanding language, and perceiving environments."

Instead of being explicitly programmed with rigid rules for every scenario, AI systems use data and algorithms to "learn" how to make decisions or predictions.

---

### 1. How Does AI Work?
AI works by combining large amounts of data with fast, iterative processing and intelligent algorithms. This allows the software to learn automatically from patterns or features in the data.

The core engine driving modern AI is "Machine Learning (ML)", which has a major subset called "Deep Learning":
* "Machine Learning (ML)": Instead of writing code with every rule, you feed a computer data and let it figure out the rules. For example, by showing a computer thousands of pictures of cats and dogs, it learns to tell them apart.
* "Deep Learning (DL)": A advanced type of ML inspired by the structure of the human brain. It uses "neural networks" with many layers to solve highly complex problems, like facial recognition or translating languages in real-time.

---

### 2. The Types of AI
AI is generally categorized into three levels of capability, though only the first one exists today:

#### A. Narrow AI (or "Weak" AI)
This is the AI we use today. It is designed and trained for a "specific task".
* "Examples": Apple's Siri, Google Translate, self-driving cars, and recommendation algorithms on Netflix or Spotify.
* "Why it's "narrow"": A chess-playing AI cannot drive a car; a medical diagnosis AI cannot write a poem.

#### B. Artificial General AI (AGI or "Strong" AI)
This is hypothetical. AGI would have "human-level intelligence" across a wide range of cognitive tasks. An AGI could learn, reason, adapt, and apply knowledge to solve any problem, just like a human can. We are not there yet.

#### C. Artificial Superintelligence (ASI)
This is a future concept where AI becomes vastly smarter than the entire human race combined, possessing social skills, general wisdom, and scientific creativity. This is currently the subject of science fiction and philosophical debate.

---
```

27 PROGRAM

```
from google import genai
```

```
# Gemini API key
```

```
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"
```

```
client = genai.Client(api_key=API_KEY)
```

```
# Get prompt from user
```

```

prompt = input("Enter your prompt: ")

# Generate response

response = client.models.generate_content(

    model="gemini-3.5-flash",

    contents=prompt

)

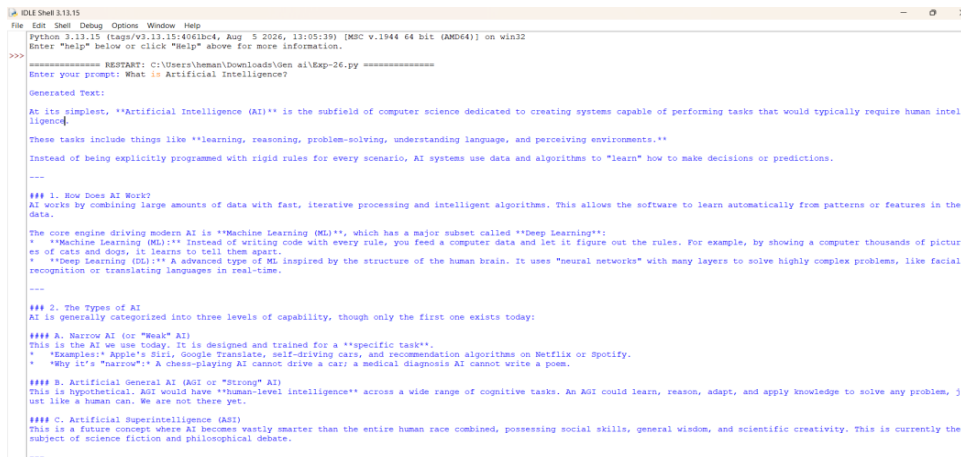
# Display output

print("\nGenerated Output:")

print(response.text)

```

OUTPUT



```

Python 3.13.15 (tags/v3.13.15:406b4c4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "help" above for more information.
>>>
===== RESTART: C:\Users\heman\downloads\Gen ai\Exp-26.py =====
Enter your prompt: What is Artificial Intelligence?

Generated Text:

At its simplest, "Artificial Intelligence (AI)" is the subfield of computer science dedicated to creating systems capable of performing tasks that would typically require human intelligence.

These tasks include things like "learning, reasoning, problem-solving, understanding language, and perceiving environments."

Instead of being explicitly programmed with rigid rules for every scenario, AI systems use data and algorithms to "learn" how to make decisions or predictions.

---

### 1. How Does AI Work?
AI works by combining large amounts of data with fast, iterative processing and intelligent algorithms. This allows the software to learn automatically from patterns or features in the data.

The core engine driving modern AI is "Machine Learning (ML)", which has a major subset called "Deep Learning":
* "Machine Learning (ML)": Instead of writing code with every rule, you feed a computer data and let it figure out the rules. For example, by showing a computer thousands of pictures of cats and dogs, it learns to tell them apart.
* "Deep Learning (DL)": A advanced type of ML inspired by the structure of the human brain. It uses "neural networks" with many layers to solve highly complex problems, like facial recognition or translating languages in real-time.

---

### 2. The Types of AI
AI is generally categorized into three levels of capability, though only the first one exists today:

#### A. Narrow AI (or "Weak" AI)
This is the AI we use today. It is designed and trained for a "specific task".
* "Examples": Apple's Siri, Google Translate, self-driving cars, and recommendation algorithms on Netflix or Spotify.
* "Why it's 'Narrow'": A chess-playing AI cannot drive a car; a medical diagnosis AI cannot write a poem.

#### B. Artificial General AI (AGI or "Strong" AI)
This is hypothetical. AGI would have "human-level intelligence" across a wide range of cognitive tasks. An AGI could learn, reason, adapt, and apply knowledge to solve any problem, just like a human can. We are not there yet.

#### C. Artificial Superintelligence (ASI)
This is a future concept where AI becomes vastly smarter than the entire human race combined, possessing social skills, general wisdom, and scientific creativity. This is currently the subject of science fiction and philosophical debate.

---

```

PROGRAM

```

from huggingface_hub import InferenceClient

# Your Hugging Face token

HF_TOKEN = "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"

# Create client

client = InferenceClient(

    provider="auto",

```

```

    api_key=HF_TOKEN
)

# Get user prompt
prompt = input("Enter your prompt: ")

# Generate text
response = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V3-0324",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ],
    max_tokens=200
)

```

```

print("\n==== GENERATED TEXT =====")
print(response.choices[0].message.content)

```

OUTPUT

```

C:\Users\user> python ai/vb-25.py
Python 3.11.10 (tags/v3.11.10:4d00353, Aug 5 2024, 13:05:39) [AMD64] on win32
Enter "help" below or click "help" above for more information.
Enter your prompt: What is database in short words

===== GENERATED TEXT =====
A "database" is an organized collection of data stored and accessed electronically. It allows users to "store, retrieve, update, and manage" information efficiently.

*** Simple Example: ***
Think of it like a "digital filing cabinet" where information (like names, orders, or products) is stored in tables for quick access.

Common types include "SQL" (structured, like MySQL) and "NoSQL" (flexible, like MongoDB).

**Shortest Definition:**
"A system for storing and managing structured data."

```

29 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

problem = input("Enter the computational problem: ")

prompt = f"""

You are a Python programming expert.

TASK:

Generate a Python program for the following problem:

{problem}

REQUIREMENTS:

1. Use Python.
2. Keep the code simple.
3. Add comments explaining important steps.
4. Use meaningful variable names.
5. The program should accept user input if required.
6. Provide the expected output format.

OUTPUT FORMAT:

1. Problem Explanation
2. Python Code
3. Sample Input
4. Sample Output

"""

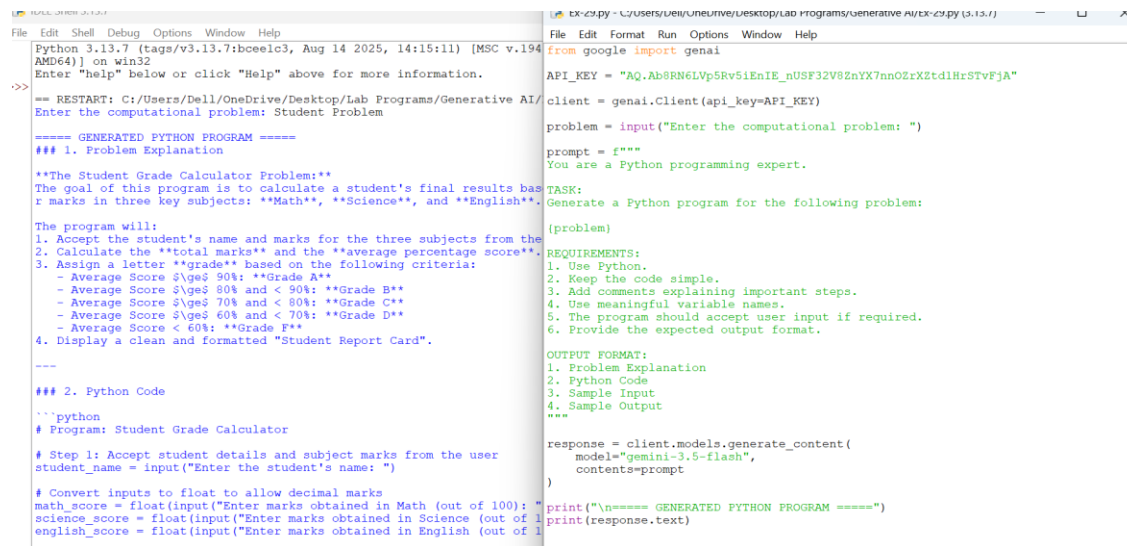
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
```

)

```
print("\n==== GENERATED PYTHON PROGRAM =====")
```

```
print(response.text)
```

OUTPUT



```
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.194
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/
Enter the computational problem: Student Problem

===== GENERATED PYTHON PROGRAM =====
### 1. Problem Explanation

**The Student Grade Calculator Problem:**
The goal of this program is to calculate a student's final results bas
r marks in three key subjects: **Math**, **Science**, and **English**.

The program will:
1. Accept the student's name and marks for the three subjects from the
2. Calculate the **total marks** and the **average percentage score**.
3. Assign a letter **grade** based on the following criteria:
    - Average Score  $\geq 90\%$ : **Grade A**
    - Average Score  $\geq 80\%$  and  $< 90\%$ : **Grade B**
    - Average Score  $\geq 70\%$  and  $< 80\%$ : **Grade C**
    - Average Score  $\geq 60\%$  and  $< 70\%$ : **Grade D**
    - Average Score  $< 60\%$ : **Grade F**
4. Display a clean and formatted "Student Report Card".

=====

### 2. Python Code

```python
Program: Student Grade Calculator

Step 1: Accept student details and subject marks from the user
student_name = input("Enter the student's name: ")

Convert inputs to float to allow decimal marks
math_score = float(input("Enter marks obtained in Math (out of 100): ")
science_score = float(input("Enter marks obtained in Science (out of 1
english_score = float(input("Enter marks obtained in English (out of 1

tx-cz.py - C:/Users/ueu/neu/neu/Desktop/Lab Programs/Generative AI/tx-cz.py (3.13.7)
File Edit Format Run Options Window Help
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrStvFjA"

client = genai.Client(api_key=API_KEY)

problem = input("Enter the computational problem: ")

prompt = f"""
You are a Python programming expert.

TASK:
Generate a Python program for the following problem:

{problem}

REQUIREMENTS:
1. Use Python.
2. Keep the code simple.
3. Add comments explaining important steps.
4. Use meaningful variable names.
5. The program should accept user input if required.
6. Provide the expected output format.

OUTPUT FORMAT:
1. Problem Explanation
2. Python Code
3. Sample Input
4. Sample Output
"""

response = client.models.generate_content(
 model="gemini-3.5-flash",
 contents=prompt
)

print("\n==== GENERATED PYTHON PROGRAM =====")
print(response.text)
```

## 30 PROGRAM

```
from google import genai
```

```
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrStvFjA"
```

```
client = genai.Client(api_key=API_KEY)
```

```
schema = """
```

```
Database: CollegeDB
```

```
Table: Students
```

```
Columns:
```

```
student_id INT
```

```
name VARCHAR(100)
```

```
age INT
```

```
department VARCHAR(50)
```

```

marks INT
"""

requirement = input("Enter your SQL requirement: ")

prompt = f"""
You are an SQL expert.

DATABASE SCHEMA:

{schema}

REQUIREMENT:

{requirement}

INSTRUCTIONS:

1. Generate the appropriate SQL query.
2. Use only the tables and columns provided.
3. Do not invent columns.
4. Keep the query simple and correct.
5. Explain the query briefly.

OUTPUT FORMAT:

SQL QUERY:

<SQL query>

EXPLANATION:

<short explanation>
"""

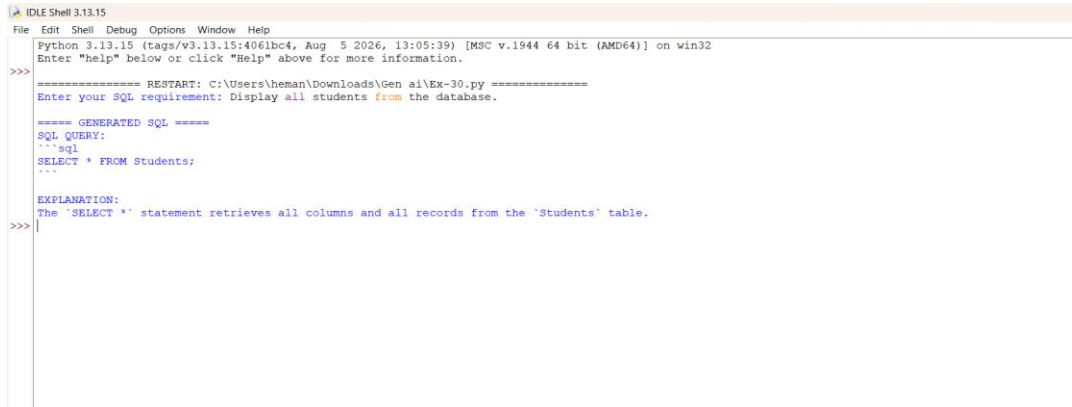
response = client.models.generate_content(
 model="gemini-3.5-flash",
 contents=prompt
)

print("\n===== GENERATED SQL =====")

```

```
print(response.text)
```

## OUTPUT



```
IDLE Shell 3.13.15
File Edit Shell Debug Options Window Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\heman\Downloads\Gen ai\Ex-30.py =====
Enter your SQL requirement: Display all students from the database.

===== GENERATED SQL =====
SQL QUERY:
'''sql
SELECT * FROM Students;
'''

EXPLANATION:
The `SELECT *` statement retrieves all columns and all records from the `Students` table.
>>>
```

## 31 PROGRAM

```
from google import genai
```

```
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"
```

```
client = genai.Client(api_key=API_KEY)
```

```
def generate(prompt):
```

```
 response = client.models.generate_content(
```

```
 model="gemini-3.5-flash",
```

```
 contents=prompt
```

```
)
```

```
 return response.text
```

```
task = """
```

```
Explain the benefits of Artificial Intelligence in education
```

```
in a short paragraph.
```

```
"""
```

```
ZERO-SHOT
```

```
zero_shot = f'"""
```

```
{task}
```



Use simple and clear language.

"""

# ONE-SHOT

one\_shot = f"""

{task}

Example:

Topic: Benefits of Computers in Education

Example Response:

Computers help students access information quickly, learn through interactive materials, complete assignments, and improve digital skills.

Now answer the given task in a similar style.

"""

# FEW-SHOT

few\_shot = f"""

{task}

Example 1

Topic: Benefits of Computers in Education

Response:

Computers help students access information quickly, use educational software, complete assignments, and develop digital skills.

Example 2:

Topic: Benefits of Online Learning

Response:

Online learning provides flexible access to educational materials, allows students to learn at their own pace, and supports learning

from different locations.

Now answer the task using the same style and structure.

```
"""
```

```
print("\n===== ZERO-SHOT =====")
result_zero = generate(zero_shot)
print(result_zero)
print("\n===== ONE-SHOT =====")
result_one = generate(one_shot)
print(result_one)
print("\n===== FEW-SHOT =====")
result_few = generate(few_shot)
print(result_few)
Comparison
print("\n===== COMPARISON =====")
```

```
print("""
Criteria Zero-shot One-shot Few-shot

Quality Good Very Good Excellent
Relevance Good Very Good Excellent
Accuracy Good Very Good Excellent
Consistency Moderate Good Very Good

```

Conclusion:

Zero-shot prompting is simple and requires no examples.

One-shot prompting improves the response by providing one example.

Few-shot prompting generally provides the most consistent response because multiple examples guide the model.

""")

## OUTPUT

```
Python 3.13.7 (tags/v3.13.7:1b0ee13, Aug 14 2025, 14:15:11) [MSC v.1916] on win32
Enter "help" below or click "Help" above for more information.
>>> == RESTART: C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/Ex-31.py (3.13.7)

----- ZERO-SHOT -----
Artificial Intelligence (AI) makes learning easier and more personalized. For students, AI acts like a 24/7 tutor, explaining difficult topics at their own comfortable pace. For teachers, AI automatically grades assignments and handling paperwork, allowing more time on helping their students. Ultimately, AI helps make education accessible, and fun.

----- ONE-SHOT -----
Artificial Intelligence helps students learn through personalized instant tutoring support, study at their own pace, and assist in managing administrative tasks.

----- FEW-SHOT -----
Topic: Benefits of Artificial Intelligence in Education
Response:
Artificial Intelligence helps students access personalized learning, instant feedback on assignments, automates administrative tasks, and supports round-the-clock tutoring assistance.

----- COMPARISON -----
Criteria Zero-shot One-shot Few-shot

Quality Good Very Good Excellent
Relevance Good Very Good Excellent
Accuracy Good Very Good Excellent
Consistency Moderate Good Very Good

Conclusion:
Zero-shot prompting is simple and requires no examples.
One-shot prompting improves the response by providing one example.

def generate(prompt):
 response = client.models.generate_content(
 model="gemini-3.5-flash",
 contents=prompt
)
 return response.text

task = """
Explain the benefits of Artificial Intelligence in education
in a short paragraph.
"""

ZERO-SHOT
zero_shot = f"""
(task)
Use simple and clear language.
"""

ONE-SHOT
one_shot = f"""
(task)
Examples:
Topic: Benefits of Computers in Education
Example Response:
Computers help students access information quickly, learn through
interactive materials, complete assignments, and improve digital
skills.
"""
```

## 32 PROGRAM

from google import genai

API\_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE\_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api\_key=API\_KEY)

```
def generate(prompt):
 response = client.models.generate_content(
 model="gemini-3.5-flash",
 contents=prompt
)
```

```
return response.text
```

```
task = """
```

```
Explain the advantages of Artificial Intelligence in healthcare.
```

```
"""
```

```
Prompt 1
```

```
prompt1 = f"""
```

```
{task}
```

```
Write a short answer.
```

```
"""
```

```
Prompt 2
```

```
prompt2 = f"""
```

```
{task}
```

```
Explain the answer using:
```

```
1. Introduction
```

```
2. Applications
```

```
3. Benefits
```

```
4. Conclusion
```

```
Use simple language.
```

```
"""
```

# Prompt 3

prompt3 = f"""

You are an AI and healthcare expert.

TASK:

{task}

REQUIREMENTS:

- Use simple language.
- Explain at least three advantages.
- Give practical examples.
- Be accurate.
- Avoid unnecessary information.
- Use headings.
- End with a conclusion.
- Keep the answer between 150 and 200 words.

OUTPUT FORMAT:

Introduction

Advantages

Examples

Conclusion

"""

```
Generate responses
```

```
result1 = generate(prompt1)
```

```
result2 = generate(prompt2)
```

```
result3 = generate(prompt3)
```

```
print("\n===== PROMPT 1 =====")
```

```
print(result1)
```

```
print("\n===== PROMPT 2 =====")
```

```
print(result2)
```

```
print("\n===== PROMPT 3 =====")
```

```
print(result3)
```

```
Evaluation prompt
```

```
evaluation_prompt = f"""
```

```
Evaluate the following three responses.
```

```
TASK:
```

```
{task}
```

```
RESPONSE 1:
```

```
{result1}
```

RESPONSE 2:

{result2}

RESPONSE 3:

{result3}

Evaluate each response using:

1. Relevance
2. Accuracy
3. Completeness
4. Clarity
5. Format adherence

Give each criterion a score from 1 to 5

Then identify the best-performing response.

Finally, create a refined prompt that would produce an even better answer.

Use this format:

RESPONSE 1 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

RESPONSE 2 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

RESPONSE 3 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

BEST RESPONSE:

...

REFINED PROMPT:

...

"

evaluation = generate(evaluation\_prompt)

print("\n===== EVALUATION =====")

print(evaluation)

OUTPUT

```
IDE Shell 3.13.7
Python 3.13.7 (tags/v3.13.7:1bceec1c, Aug 14 2025, 14:15:11) [MSC v.1916] on win32
Enter "help" below or click "Help" above for more information.
>>> == RESTART: C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/Ex-31.py (3.13.7) ==

===== ZERO-SHOT =====
Artificial Intelligence (AI) makes learning easier and more personalized for everyone. For students, AI acts like a 24/7 tutor, explaining difficult concepts and letting them learn at their own comfortable pace. For teachers, AI automatically grades assignments and handles paperwork, allowing them to focus more on helping their students. Ultimately, AI helps make education more effective, accessible, and fun.

===== ONE-SHOT =====
Artificial Intelligence helps students learn through personalized learning. It provides instant tutoring support, study at their own pace, and assists teachers in managing administrative tasks.

===== FEW-SHOT =====
Topic: Benefits of Artificial Intelligence in Education
Response: Artificial Intelligence helps students access personalized learning, receive instant feedback on assignments, automate administrative tasks, and supports round-the-clock tutoring assistance.

===== COMPARISON =====
Criteria Zero-shot One-shot Few-shot

Quality Good Very Good Excellent
Relevance Good Very Good Excellent
Accuracy Good Very Good Excellent
Consistency Moderate Good Very Good

Conclusion:
Zero-shot prompting is simple and requires no examples.
One-shot prompting improves the response by providing one example.

Ex-31.py - C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/Ex-31.py (3.13.7)
File Edit Format Run Options Window Help
from google import genai
API_KEY = "AQ.AB8P6LWp5Wv5LEaIE_g0SF3ZV82nYX7ao0EzXt-d1RcSTvF3A"
client = genai.Client(api_key=API_KEY)

def generate(prompt):
 response = client.models.generate_content(
 model="gemini-3.5-flash",
 content=prompt
)
 return response.text

task = """
Explain the benefits of Artificial Intelligence in education
in a short paragraph.
"""

ZERO-SHOT
zero_shot = """
(task)
Use simple and clear language.
"""

ONE-SHOT
one_shot = """
(task)
Example:
Topic: Benefits of Computers in Education
Example Response:
Computers help students access information quickly, learn through interactive materials, complete assignments, and improve digital skills.
"""
```



### 33 PROGRAM

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "google/flan-t5-base"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

print("Engineering College AI Chatbot")

print("Type 'exit' to stop.\n")

while True:

 question = input("Student: ")

 if question.lower() == "exit":

 print("Chatbot: Goodbye!")

 break

 prompt = f"""

 You are an engineering college assistant.

 Answer the student's question clearly and briefly.

 Student question: {question}

 """

 inputs = tokenizer(

 prompt,

 return_tensors="pt"

)

 outputs = model.generate(

 **inputs,

 max_new_tokens=100

)
```

```

answer = tokenizer.decode(
 outputs[0],
 skip_special_tokens=True
)

print("Bot:", answer)

print()

```

## OUTPUT

```

Engineering Technical Support Chatbot
Type 'exit' to stop.

Student: What is a semester?
Bot: a year

Student: What is Artificial intelligence
Bot: artificial intelligence is a computer program that learns information from a computer.

```

## 34 PROGRAM

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "google/flan-t5-base"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

print("Engineering Technical Support Chatbot")

print("Type exit to stop.\n")

while True:

 question = input("Engineer: ")

 if question.lower() == "exit":

 print("Bot: Goodbye!")

 break

 prompt = f""

```

You are an engineering technical support assistant.

Explain the engineering problem and provide a possible solution.

Problem: {question}

Answer:

```
"""
```

```
inputs = tokenizer(
 prompt,
 return_tensors="pt",
 truncation=True
)

outputs = model.generate(
 **inputs,
 max_new_tokens=150
)

answer = tokenizer.decode(
 outputs[0],
 skip_special_tokens=True
)

print("\nTechnical Solution:")

print(answer)

print()
```

## OUTPUT

```
Python 3.11.7 (tags/v3.11.7:4200b44, Aug 9 2023; 13:07:39) [AMD V.1944 64 bit (AMD64)] on win32
File "help" below or click "help" above for more information.
>>>
=====
Warning: You are sending unauthenticated requests to the HF Hub. Please set a token to enable higher rate limits and faster downloads.
Training weights: hf://openai/gpt2-medium, hf://openai/gpt2-medium, hf://openai/gpt2-medium, hf://openai/gpt2-medium, hf://openai/gpt2-medium
(Warning: The file weights mapping and config for this model specifies to the model weights to be loaded, but both are present in the checkpoints with different values, so we
will not use them. You should update the config with the correct embeddings file to silence this warning.)
Engineering Technical Support Chatbot
Type Ctrl+C to stop.
Engineer: why does a computer overheat?
Technical Solution:
How long
Engineer: What causes voltage fluctuation in an electrical system?
Technical Solution:
Electrical current
Engineer: |
```

### 35 PROGRAM

```
from diffusers import StableDiffusionPipeline

import torch

model_id = "runwayml/stable-diffusion-v1-5"

pipe = StableDiffusionPipeline.from_pretrained(
 model_id,
 torch_dtype=torch.float32
)

pipe = pipe.to("cpu")

prompts = [
 "A modern suspension bridge",
 "A modern suspension bridge at sunset",
 "A futuristic suspension bridge with smart sensors",
 "A suspension bridge designed in a cyberpunk style"
]

for i, prompt in enumerate(prompts, 1):
 print(f"Generating image {i}...")
 image = pipe(prompt).images[0]
 filename = f"bridge_prompt_{i}.png"
 image.save(filename)
 print("Saved:", filename)

print("\nAll images generated.")
```

## OUTPUT

```
1)
2
3 pipe = pipe.to("cpu")
4
5 prompt = input("Enter engineering image prompt: ")
6
7 print("\nGenerating image...")
8
9 image = pipe(
10 prompt,
11 num_inference_steps=20
12).images[0]
13
14 image.save("engineering_image.png")
15
16 print("Image saved as engineering_image.png")
17
```

blems Output Debug Console Terminal Ports Spell Checker Filter (e.g. text, excludeText, t... Code

loading pipeline components...: 0%| | 0/7 [00:00<?, ?it/s]

loading weights: 0%| | 0/196 [00:00<?, ?it/s]esc[A

loading weights: 100%|#####| 196/196 [00:00<00:00, 4208.69it/s]

loading pipeline components...: 14%|#4 | 1/7 [00:00<00:01, 4.05it/s]

loading pipeline components...: 29%|##8 | 2/7 [00:00<00:01, 3.41it/s]

loading weights: 0%| | 0/396 [00:00<?, ?it/s]esc[A

loading weights: 100%|#####| 396/396 [00:00<00:00, 4887.93it/s]

loading pipeline components...: 43%|###2 | 3/7 [00:00<00:01, 3.73it/s]

loading pipeline components...: 86%|#####5 | 6/7 [00:00<00:00, 8.78it/s]

loading pipeline components...: 100%|#####| 7/7 [00:00<00:00, 7.35it/s]

Enter engineering image prompt: A detailed engineering blueprint of a modern suspension bridge, showing steel cables, structural supports, technical annotations, precise measurements, blue and white drafting style

## 36 PROGRAM

```
from diffusers import StableDiffusionPipeline
```

```
import torch
```

```
model_id = "runwayml/stable-diffusion-v1-5"
```

```
pipe = StableDiffusionPipeline.from_pretrained(
```

```
 model_id,
```

```
 torch_dtype=torch.float32
```

```
)
```

```
pipe = pipe.to("cpu")
```

```
prompts = [
```

```
 "A modern suspension bridge",
```

```
 "A modern suspension bridge at sunset",
```

```
]
```

```

for i, prompt in enumerate(prompts, 1):
 print(f"Generating image {i}...")
 image = pipe(prompt).images[0]
 filename = f"bridge_prompt_{i}.png"
 image.save(filename)
 print("Saved:", filename)
print("\nAll images generated.")

```

## OUTPUT



## 37 PROGRAM

```

import os
import whisper

os.environ["PATH"] += os.pathsep + r"C:\Program Files\ffmpeg\bin"

model = whisper.load_model("base")

audio_file = input("Enter audio file path: ")

result = model.transcribe(
 audio_file,

```

```

fp16=False

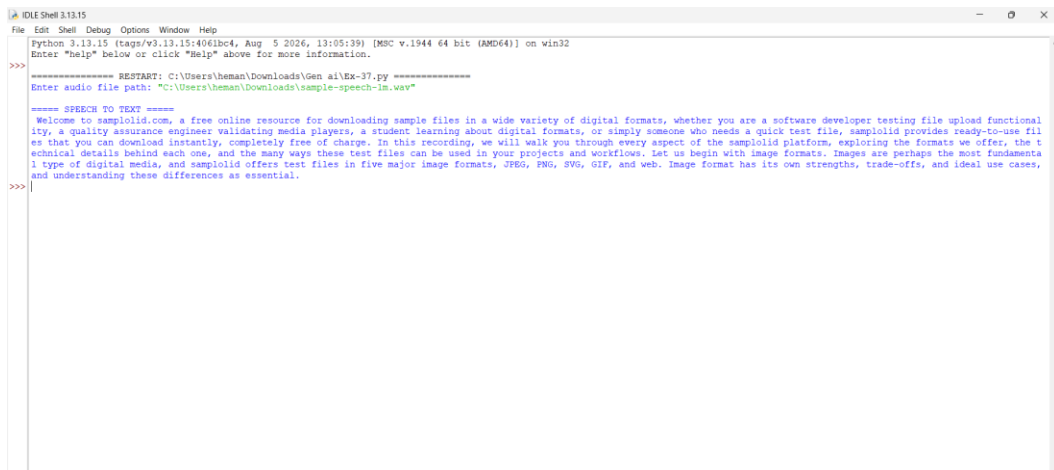
)

print("\n===== SPEECH TO TEXT =====")

print(result["text"])

```

## OUTPUT



```

IDIEShell@1315
File Edit Shell Debug Options Window Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\heman\Downloads\Gen ai\Ex-37.py =====
Enter audio file path: "C:\Users\heman\Downloads\sample-speech-in.wav"

===== SPEECH TO TEXT =====
Welcome to sampleid.com, a free online resource for downloading sample files in a wide variety of digital formats, whether you are a software developer testing file upload functionality, a quality assurance engineer validating media players, a student learning about digital formats, or simply someone who needs a quick test file, sampleid provides ready-to-use files that you can download instantly, completely free of charge. In this recording, we will walk you through every aspect of the sampleid platform, exploring the formats we offer, the technical details behind each one, and the many ways these test files can be used in your projects and workflows. Let us begin with image formats. Images are perhaps the most fundamental type of digital media, and sampleid offers test files in five major image formats, JPEG, PNG, SVG, GIF, and web. Image format has its own strengths, trade-offs, and ideal use cases, and understanding these differences is essential.
>>>

```

## 38 PROGRAM

```

import pytsx3

engine = pytsx3.init()

text = input("Enter engineering text: ")

engine.say(text)

engine.runAndWait()

print("Speech generated successfully.")

```

## OUTPUT

```
File Edit Shell Debug Options Window Help
Python 3.13.15 [tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39] [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
===== RESTART: C:\Users\heman\Downloads\Gen ai\Ex-38.py =====
Enter engineering text: HI THIS IS HEMANTH
Speech generated successfully.
>> |
```

### 39 PROGRAM

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "facebook/bart-large-cnn"

print("Loading summarization model...")

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

print("\nEnter engineering document.")

print("Type END when finished.\n")

lines = []

while True:

 line = input()

 if line.upper() == "END":

 break

 lines.append(line)

document = " ".join(lines)

if len(document.split()) < 30:

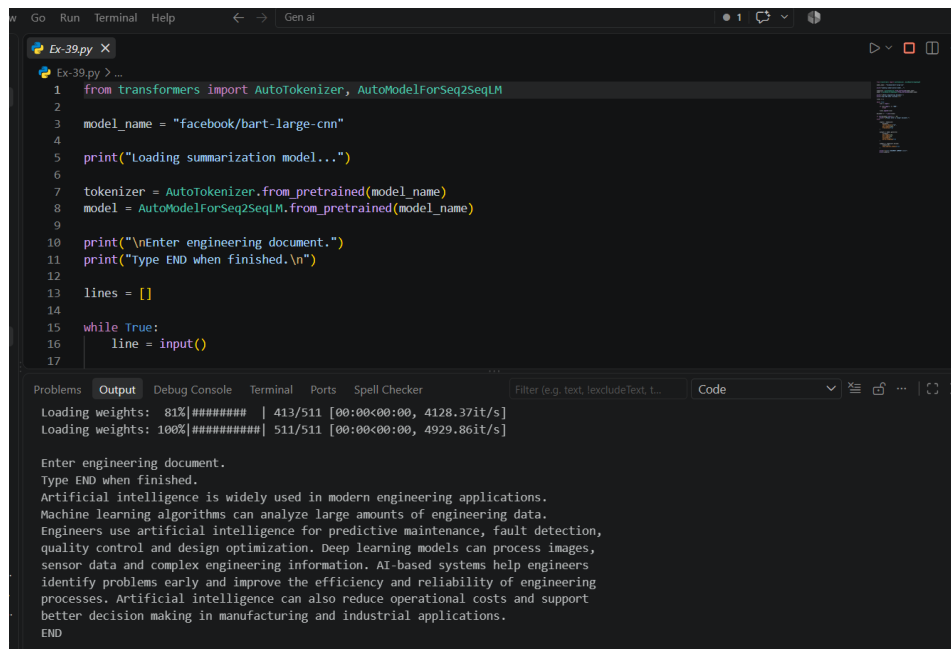
 print("\nPlease enter a longer document.")

else:
```



```
inputs = tokenizer(
 document,
 return_tensors="pt",
 max_length=1024,
 truncation=True
)
outputs = model.generate(
 **inputs,
 max_length=150,
 min_length=30,
 num_beams=4,
 early_stopping=True
)
summary = tokenizer.decode(
 outputs[0],
 skip_special_tokens=True
)
print("\n===== DOCUMENT SUMMARY =====")
print(summary)
```

## OUTPUT



```
Ex-39.py X
Ex-39.py > _
1 from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
2
3 model_name = "facebook/bart-large-cnn"
4
5 print("Loading summarization model...")
6
7 tokenizer = AutoTokenizer.from_pretrained(model_name)
8 model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
9
10 print("\nEnter engineering document.")
11 print("Type END when finished.\n")
12
13 lines = []
14
15 while True:
16 line = input()
17
Loading weights: 81%|##### | 413/511 [00:00<00:00, 4128.37it/s]
Loading weights: 100%|##### | 511/511 [00:00<00:00, 4929.86it/s]

Enter engineering document.
Type END when finished.
Artificial intelligence is widely used in modern engineering applications.
Machine learning algorithms can analyze large amounts of engineering data.
Engineers use artificial intelligence for predictive maintenance, fault detection,
quality control and design optimization. Deep learning models can process images,
sensor data and complex engineering information. AI-based systems help engineers
identify problems early and improve the efficiency and reliability of engineering
processes. Artificial intelligence can also reduce operational costs and support
better decision making in manufacturing and industrial applications.
END
```

## 40 PROGRAM

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "Helsinki-NLP/opus-mt-en-fi"

print("Loading translation model...")

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

text = input("Enter English engineering text: ")

inputs = tokenizer(
 text,
 return_tensors="pt",
 truncation=True
)

outputs = model.generate(
 **inputs,
 max_new_tokens=100
)
```

## OUTPUT

## 41 PROGRAM

```
resumes.append(resume)
```

```

job_embedding = model.encode([job])
resume_embeddings = model.encode(resumes)
scores = cosine_similarity(
 job_embedding,
 resume_embeddings
)[0]
results = []
for i in range(n):
 results.append(
 (i + 1, resumes[i], scores[i])
)
results.sort(
 key=lambda x: x[2],
 reverse=True
)
print("\n===== RESUME RANKING =====")
for rank, (number, resume, score) in enumerate(results, 1):
 print("\nRank:", rank)
 print("Resume:", number)
 print("Match Score:", round(score * 100, 2), "%")
 print("Text:", resume)

```

## OUTPUT

```
IDLE Shell 3.13.15
File Edit Shell Debug Options Window Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\heman\Downloads\Gen ai\Ex-41.py =====
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 0% | 0/103 [00:00<7, 7it/s]Loading weights: 100% [REDACTED] 103/103 [00:00<00:00, 20648.87it/s]
===== AI RESUME SCREENING =====

Enter engineering job description:
Python Machine Learning Engineer with experience in Python, machine learning, deep learning, SQL, TensorFlow and data analysis.

Enter number of resumes: 3

Enter Resume 1:
Python developer with experience in machine learning, TensorFlow, SQL and data analysis.

Enter Resume 2:
Java developer with experience in Spring Boot, Java, HTML, CSS and web development.

Enter Resume 3:
Machine learning engineer experienced in Python, deep learning, SQL and TensorFlow.

===== RESUME RANKING =====

Rank: 1
Resume: 3
Match Score: 94.2 %
Text: Machine learning engineer experienced in Python, deep learning, SQL and TensorFlow.

Rank: 2
Resume: 1
Match Score: 93.0 %
Text: Python developer with experience in machine learning, TensorFlow, SQL and data analysis.

Rank: 3
Resume: 2
Match Score: 36.44 %
Text: Java developer with experience in Spring Boot, Java, HTML, CSS and web development.

>>>
```

## 42 PROGRAM

```
from sentence_transformers import SentenceTransformer
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
print("===== AI RESUME SCREENING =====")
```

```
job = input("\nEnter engineering job description:\n")
```

```
n = int(input("\nEnter number of resumes: "))
```

```
resumes = []
```

```
for i in range(n):
```

```
 print(f"\nEnter Resume {i + 1}:")
```

```
 resume = input()
```

```
 resumes.append(resume)
```

```
job_embedding = model.encode([job])
```

```
resume_embeddings = model.encode(resumes)
```

```
scores = cosine_similarity(
```

```
 job_embedding,
```

```
 resume_embeddings
```

```

)[0]

results = []

for i in range(n):

 results.append(

 (i + 1, resumes[i], scores[i])

)

results.sort(

 key=lambda x: x[2],

 reverse=True

)

print("\n==== RESUME RANKING =====")

for rank, (number, resume, score) in enumerate(results, 1):

 print("\nRank:", rank)

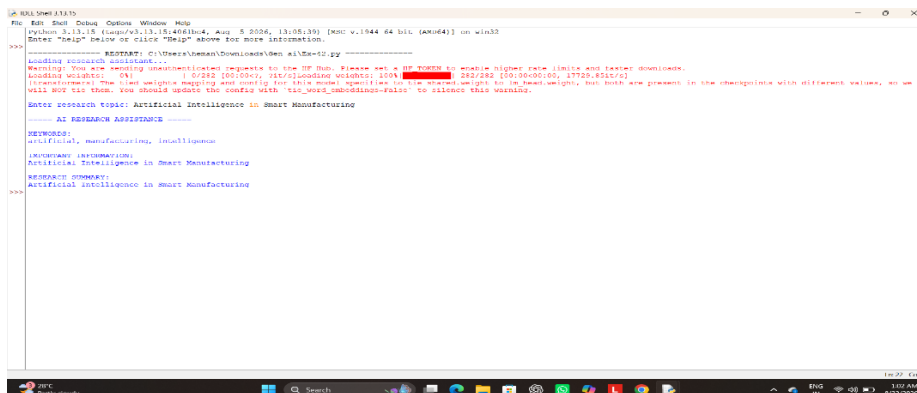
 print("Resume:", number)

 print("Match Score:", round(score * 100, 2), "%")

 print("Text:", resume)

```

## OUTPUT



```

Python 3.13.0 (tags/v3.13.0:406b444, Aug 5 2024, 13:08:35) [AMD64] on win32
Enter "help" for more information.

>>> ===== RESUME: C:\Users\user\Downloads\Gen AI\Se-02.py =====
Loading research capabilities...
Warning: You are sending unsolicited requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% 0/250 [00:00<, 153.00MB/s] 0/0 [00:00<, 17720.40KB/s]
(Transformer) The load weights mapping and saving for this model specifies to tie shared weights to lm_head.weight, but both are present in the checkpoints with different values, so we
Wait for the user, you should update the config with "tie_lm_head.weight=False" to silence this warning.
Enter research topic: Artificial Intelligence in Smart Manufacturing

===== AI RESEARCH ASSISTANCE =====
RESEARCH:
artificial, manufacturing, intelligence
IMPORTANT INFORMATION:
Artificial Intelligence in Smart Manufacturing
RESEARCH SUMMARY:
Artificial intelligence in smart manufacturing
>>>

```

